When Are Agent Execution Edits Safe? An Exact Theory of Fork, Restore, and Merge

***Abstract*—Agent runtimes can Fork an execution, Restore a checkpoint, or Merge branches without restarting a task. We call these operations *execution edits* because they change which parts of the workflow will run next. An edit changes future work without undoing a tool action that has already been authorized or released. An unsafe edit can therefore authorize the same action twice, discard a result the workflow still requires, or conflict with a call that began before the edit. Existing Agent systems support Fork, Restore, or Merge, but neither these systems nor existing synthesis methods derive a requested edit's safety conditions from the current execution record. We give an exact procedure for deciding whether an execution edit is safe. It either computes every safe way for the edited workflow to continue or proves that every possible implementation is unsafe. Given a registered workflow and execution record, a trusted controller derives the edited workflow and the outcomes that it must preserve. The procedure constructs the complete executions allowed by the workflow and policy, then removes any execution whose prefix would leave a still-required outcome with no safe completion. If no execution remains, it returns a finite, independently checkable proof that the edit has no safe implementation. Otherwise, the remaining executions contain every safe continuation, and the runtime permits exactly their prefixes. Our main theorem proves this result for six Fork, Restore, and Merge edits and later registered extensions, establishes atomic runtime enforcement, and identifies the information that any exact checker must retain.**

I. Introduction

Tool-using Agent runtimes can change an execution while it is in progress. They let an Agent Fork an execution into alternatives, Restore an earlier checkpoint, and Merge branches [1], [2], [3], [4], [5], [6]. These operations support exploration, recovery, and reuse without restarting the task. We call them *execution edits* because they change which parts of the workflow will run next.

Consider an Agent buying one laptop for a school. Before asking the school to approve the purchase, it saves checkpoint k. The school approves one purchase, and the Agent later chooses supplier R and authorizes payment. The Agent then restores k to compare supplier L and merges the L branch into the current execution. If the runtime sees only the restored workspace, it may request the same approval again or continue with L even though payment to R is already authorized [7]. Either result violates the intended workflow because the first repeats a one-time approval and the second combines incompatible supplier actions.

Detecting this failure requires more than the restored checkpoint. The execution record contains four facts that the checker needs: (1) the workflow and its required outcomes, (2) which calls refer to the same action, (3) earlier authorizations and call progress, and (4) which rules govern a call that overlaps the edit. Each field can change the correct answer. Equal workspaces, an execution trace alone, an authorization log alone, or a replacement workflow proposed by the Agent cannot reconstruct this record.

Existing approaches cover parts of this problem but do not derive its complete safety condition. Agent systems provide branching, staged effects, transactions, or recovery checks [8], [9], [10], [11], [12], [13], [14], [15]. Control and synthesis choose behavior after a model of possible executions and desired final states has been supplied [16], [17], [18], [19]. Neither line of work derives from a recorded execution everything that a particular edit must preserve before it takes effect.

Our key idea is that an execution edit must preserve both past actions and still-required outcomes. Every outcome required before the edit must remain required, already be satisfied by the recorded execution, or be explicitly removed by the same authority that required it. The Agent identifies a registered edit. The platform authenticates the registered workflow, and the trusted controller derives the edited workflow and inherited requirements from that record rather than from Agent text. The Agent therefore cannot obtain a favorable answer by submitting a replacement workflow that hides inconvenient facts.

This idea creates three technical problems. First, calls copied by Fork or Restore may refer to the same underlying action, so the controller must distinguish a new authorization from reuse of an earlier authorization and return value. Second, after every allowed partial execution, every outcome that is still required and still possible must retain a safe way to finish. The checker must enforce this condition without forbidding additional safe behavior. Third, the answer must take effect atomically with calls that overlap the edit and remain valid if the runtime or machine stops unexpectedly and later recovers.

We give an exact checker that either computes every safe way for an edited workflow to continue or proves that every possible implementation is unsafe. For a requested edit, the controller uses the registered workflow and current execution record to construct every allowed complete execution. It removes an execution whenever one of its partial executions would leave a still-required outcome with no safe completion. If no execution remains, the controller returns an independently checkable proof that no safe implementation exists. Otherwise, the remaining executions contain every safe way for the edit to proceed. Before the edit takes effect, the runtime checks the current execution record again and atomically switches to the newly computed rules. Every overlapping call is therefore

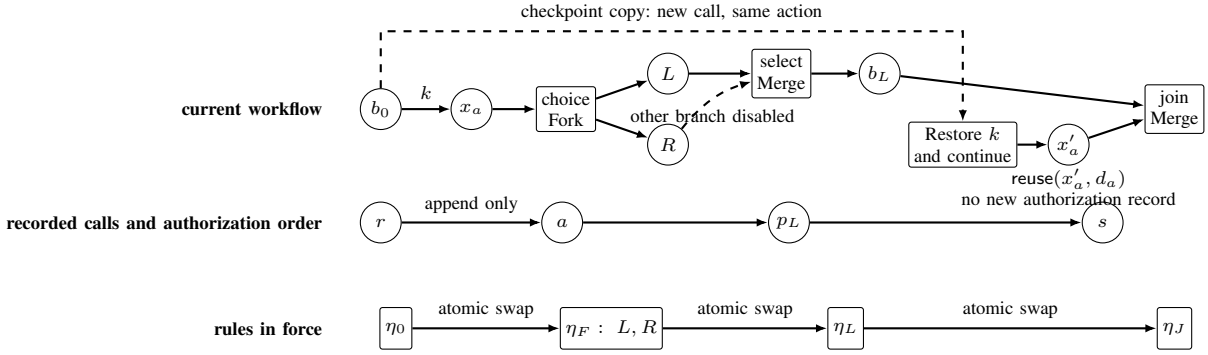

Fig. 1. Fork, Restore, and Merge change the current workflow but do not erase recorded calls. Restore creates a new workflow call that refers to the same action as the original call. The atomic rule update orders both creation and reuse of an authorization record against the rule change.

checked entirely under either the old rules or the new rules. Figure 1 illustrates this process for an approval workflow containing Fork, Merge, Restore, and calls that overlap the atomic rule update. Section II derives the example step by step. After Restore, the repeated approval call reuses the earlier authorization record instead of authorizing the approval again.

a) Contributions:

1) **Safety model.** We define safety for Agent Fork, Restore, and Merge from the execution record rather than from a replacement workflow proposed by the Agent. The model preserves past actions and every still-required outcome, and it proves that every exact checker needs the four facts identified above.
2) **Exact checking and runtime implementation.** We construct an executable checker that returns either every safe way for the edit to proceed or a proof that none exists. For all six registered edits and later registered extensions, the checker accepts exactly when a safe runtime implementation exists. The runtime puts the result into effect atomically, preserves safety across repeated edits and recovery, and has the same Agent-visible behavior as an ideal atomic machine.
3) **Mechanization and executable validation.** Six Lean modules mechanize registration, all six edit rules, the atomic rule update, calls concurrent with that update, and preservation across finite executions. Nineteen paper-specific tests and measurements over 2–128 outcomes exercise the executable checker.

II. Execution-Edit Safety

Imagine an Agent buying one laptop for a school. It reserves the budget, saves checkpoint k, receives one purchase approval, and then compares suppliers L and R. A careless Restore or Merge could request approval twice, pay both suppliers, or permit shipment before the selected supplier is paid. A safe execution approves the purchase once, pays exactly one supplier, and permits shipment only after that payment.

We formalize this story as an approval workflow that authorizes at most one order. The Agent reserves the budget and saves checkpoint k before approval. The approval is then authorized once, creating an authorization record that Restore does not delete. The Agent next requests a Fork between suppliers L and R. The workflow allows two outcomes: order from L or order from R. In either outcome, payment authorization must precede shipment authorization:

$$\begin{aligned} o_L &: \; x_L^p \prec x_L^s, \\ o_R &: \; x_R^p \prec x_R^s, \end{aligned} \tag{1}$$

where x_i^p and x_i^s are payment and shipment calls. A *workflow call* x is one place where the workflow can invoke a protected tool. The function $q_{\mathrm{act}}(x) = d$ records which external action the call refers to. Calls that refer to the same action share one authorization record.

In this example, both shipment calls authorize the same one-order shipment:

$$q_{\mathrm{act}}(x_L^s) = q_{\mathrm{act}}(x_R^s) = d_s.$$

The two payment calls instead refer to different actions d_L^p and d_R^p.

The authorization log Δ records newly authorized actions in order. The notation $\mathsf{lab}(\Delta)$ keeps only their authorization labels. Budget reservation contributes r, approval contributes a, the two payments contribute p_L and p_R, and shipment contributes s. At the Fork request, $\mathsf{lab}(\Delta) = ra$. The policy allows prefixes of rap_Ls and rap_Rs, but never both payments or a payment before approval. Although the supplier outcomes are exclusive, both must still have a safe way to finish when the Fork is checked.

A. Accepted and Rejected Forks

The Agent sends $u = \mathsf{ForkChoice}(b, \sigma)$, which names a current branch and a registered edit rule. The trusted controller constructs the edited workflow from the execution record. It also derives the outcomes that remain required and the action referred to by every workflow call.

Write M_o for the ways to complete outcome o while obeying the policy. Each outcome has one allowed call order:

$$\begin{aligned} z_L &= \mathsf{new}(x_L^p, d_L^p)\mathsf{new}(x_L^s, d_s), \\ z_R &= \mathsf{new}(x_R^p, d_R^p)\mathsf{new}(x_R^s, d_s). \end{aligned}$$

Thus $M_{o_L} = \{z_L\}$ and $M_{o_R} = \{z_R\}$. These sequences contain the calls after the edit, while the existing log contributes the earlier prefix ra. Each supplier outcome is preserved, and every permitted prefix can still finish safely. The verdict is therefore *Accept* because both supplier outcomes remain possible, every permitted prefix is policy safe, and each such prefix can still reach a complete outcome. If policy excludes rap_Rs, then $M_{o_R} = \varnothing$, and the Fork is rejected because pruning supplier R would change its meaning. Rejection returns a proof listing the order in which completions were removed and the reason for each removal, rather than letting the Agent weaken the requested edit. The verdict in this variant is *Reject* because checking only the surviving L branch would silently weaken the requested Fork.

B. Why Safe Endpoints Are Not Enough

The previous rejection is visible at the start because one outcome has no safe completion. A harder failure occurs when every outcome is initially safe in isolation, but no single set of runtime rules can keep all of them completable. Let X, Y, Z be single-call contracts with authorization labels x, y, z, respectively, and consider

$$X \otimes (Y \oplus Z) \qquad \text{under} \qquad \mathcal{A} = \mathsf{Pref}\{yx, xz\}.$$

The two outcomes have safe complete traces yx and xz. However, executing the shared x call first keeps the $X \otimes Y$ outcome possible while leaving only the forbidden continuation xy. Allowing x is therefore unsafe, while forbidding x destroys the $X \otimes Z$ outcome. The exact verdict is *Reject*, even though an outcome-by-outcome check would accept. Section IV detects this failure by repeatedly removing executions that create such a bad prefix.

C. Calls During the Atomic Rule Update

Figure 1 separates the current workflow, the authorization log, and the rules that must change atomically.

a) New authorization during the rule update: After the shared approval and before either payment, the Agent requests $\mathsf{MergeSelect}(g, L, \sigma)$. If the payment call x^p_R from the old supplier-R branch is authorized first, it selects R and changes $\mathsf{lab}(\Delta)$ from ra to rap_R. The controller must therefore check the Merge again and reject it. If the Merge rule update occurs first, the old rules stop applying and every current call receives a new authorization token before any later x^p_R can be authorized. Leaving the old rules active would allow both payments.

b) Authorization reuse during the rule update: Now consider the Restore that keeps both branches in figure 1. A later Restore of k creates a new approval call x'_a, but both approval calls refer to the same action d_a. The restored call is new, while the underlying action remains d_a. Because d_a already has an authorization record, executing x'_a reuses that record. The restored branch therefore advances past approval without creating another authorization record. A concurrent Merge or Restore result computed before that progress is now out of date even though the authorization log has not changed. The final check must therefore compare both the authorization log and the ordered record χ of executed workflow calls. For this RestoreLive request, the checker either puts the newly computed rules into effect or returns a rejection proof.

Together, these executions give the paper's end-to-end safety rule. An edit satisfies this rule when (1) it preserves every outcome still required by the execution record, (2) every allowed step obeys policy and can still reach a complete execution, (3) every required outcome that remains possible after those steps still has a safe way to finish, and (4) the new rules and authorization tokens take effect atomically with concurrent calls. Section III states this rule precisely, and section IV constructs the exact checker.

III. Model and Security Objective

The model leaves the Agent's internal reasoning unrestricted and places every protected tool call under trusted runtime control. Each call identifies its registered place in the workflow, presents an authorization token, and submits a concrete tool request. The runtime either creates a new authorization, reuses an authorization already recorded in the log, or rejects the call. Calls governed by one policy, one authorization log, and one atomic rule change form a *policy domain*. Policy domains with disjoint actions, logs, and policies compose independently. Every individual state is finite, while the theory covers executions of arbitrary finite length. The *execution record* stores the workflow and checkpoints, which calls refer to the same action, earlier authorizations and call progress, queued requests, and the rules currently in force. The trusted runtime protects this record from Agent modification.

A. Workflow outcomes and call order

a) Basic objects: We capitalize *Agent* for the untrusted principal requesting execution edits. An *workflow call* x is one place where the workflow can invoke a protected tool. The function $q_{\mathrm{act}}(x) = d$ records the external action referred to by that call. Several workflow calls may refer to the same action d, in which case they share one authorization record. The first authorized call creates that record, and calls copied by Fork or Restore later reuse it. The execution record also stores where every copied workflow region came from. T, Δ, χ denote the current workflow, authorization log, and ordered record of processed workflow calls.

A pomset $p = (X_p, \prec_p)$ is a finite set of workflow calls with a strict partial order. Its linearizations $\mathsf{Lin}(p)$ are the words containing each member of X_p once and respecting $\prec_p$. A *workflow contract* $\mathcal{C} = (O, \{p_o\}_{o \in O})$ assigns one pomset to each member of a finite nonempty outcome set O. Each index o names one allowed completion outcome. Two outcomes remain distinct even if the calls still required for them later become identical. Write $X_{\mathcal{C}} = \bigcup_{o \in O} X_{p_o}$ and $\mathsf{CL}(\mathcal{C}) = \bigcup_{o \in O} \mathsf{Lin}(p_o)$. We require the runtime to distinguish completion from continuation.

$$v, w \in \mathsf{CL}(\mathcal{C}) \ \wedge \ v \preceq_{\mathrm{pref}} w \quad \Longrightarrow \quad v = w.$$

This condition permits incomparable traces and different outcomes with the same trace, so it does not require deterministic

outcomes. It requires a registered workflow call whenever continuing after a shared partial execution performs more work. The interface represents a choice between stopping and continuing by an authenticated terminal workflow call handled by the controller.

Contracts use the three partial constructors below. Each constructor requires operands with disjoint workflow calls and preserves the condition that no complete call sequence is a strict prefix of another.

$$\begin{aligned} O_{\mathcal{C}\oplus\mathcal{D}} &= (\{L\}\times O_{\mathcal{C}}) \uplus (\{R\}\times O_{\mathcal{D}}),\\ p_{(L,o)} &= p_o, \qquad p_{(R,v)} = p_v,\\ O_{\mathcal{C}\otimes\mathcal{D}} &= O_{\mathcal{C}}\times O_{\mathcal{D}},\\ p_{(o,v)} &= p_o \uplus p_v,\\ O_{\mathcal{C}\triangleright\mathcal{D}} &= O_{\mathcal{C}}\times O_{\mathcal{D}},\\ p_{(o,v)} &= p_o \triangleright p_v. \end{aligned} \tag{2}$$

Before adding a registered or copied operand, the controller deterministically assigns fresh workflow-call IDs and extends ρ while retaining the authenticated actions. Well-formedness already makes carried operands disjoint. In equation (2), $\uplus$ adds no cross-order and $\triangleright$ orders every event of p before every event of q, while choice uses a tagged union of outcome indices and parallel and sequence use Cartesian products. Consequently, choice retains each arm's full indexed family, parallel pairs outcomes while permitting every causal linearization, and sequence adds a barrier. After call prefix w, $\mathcal{C}/w$ retains each outcome whose pomset has a linearization prefixed by w, then removes the workflow calls in w and their order edges. The expression is undefined if no outcome remains. This operational "remaining contract" preserves outcome and workflow call identity.

Lemma 1 (Completion remains unambiguous)**.** *For every defined $\mathcal{C}/w$, no complete sequence is a strict prefix of another, and either every retained pomset is empty or none is empty.*

Proof. If a remaining suffix v strictly prefixes v', then wv strictly prefixes wv' in $\mathsf{CL}(\mathcal{C})$. If one retained pomset is empty and another is nonempty, choose a nonempty linearization v of the latter. Then w strictly prefixes wv. Both contradict the condition that no complete sequence strictly prefixes another. □

B. Execution Record

1) Recorded structure and current workflow: The workflow part of the execution record is

$$H = (\omega, G, T, K, \Sigma_\zeta, \rho, \beta, \chi, \mathsf{ver}, \eta). \tag{3}$$

ω identifies the policy domain, and η identifies the version of the rules currently in force. G records the finite sequence of workflow versions. Its node IDs, registered contracts, signed outcome requirements, and Fork, Restore, and Merge records do not change. Each branch node b carries an immutable registered contract Γ_b. Its ordered progress record χ_b lists the workflow calls already processed on that branch. The immutable records form a directed acyclic graph (DAG) whose edges record typed Fork, Restore, and Merge operations. T is the current workflow.

$$T ::= b{:}\mathcal{C} \mid T \oplus_g^s T \mid T \parallel_g T \mid \mathsf{join}(T,T) \mid T;T.$$

$s \in \{\mathsf{open}, L, R\}$ records an unresolved choice or the arm that made recorded progress, and g groups exclusive or jointly active siblings. $[\![T]\!]$ uses both arms of an open choice and only the selected arm thereafter. It interprets parallel and join with $\otimes$, and sequence with $\triangleright$. The join marker records a completed Merge and prevents the same group from being merged again.

T is complete when every pomset in $[\![T]\!]$ is empty. Lemma 1 ensures that the remaining workflow cannot contain both empty and nonempty pomsets. Set $\mathsf{heads}(b{:}\mathcal{C}) = \{b\}$. An open choice makes both arms current, a selected choice makes its winner current, parallel and join make both arms current, and sequence makes its left operand current until completion and then its right. Completing a workflow call updates the work remaining at its leaf without deleting the recorded structure. Because sequence checks complete, it retains completed outcomes from its left operand, so an isolated completed leaf remains available for a later Fork or Restore. Joining removes g immediately but keeps its continuation behind a barrier until both arms complete. A workflow call is enabled when it is minimal in some pomset of $[\![T]\!]$. Disjoint workflow calls and unique context decomposition give every enabled x a unique current leaf $\mathsf{leaf}_T(x) = b$. For $a \in \{\mathsf{new}(x,d), \mathsf{reuse}(x,d)\}$, define the paired execution update

$$\mathsf{HStep}((G,T),a) = \big(G \parallel (b,a),\, T/(x,a)\big), \qquad b = \mathsf{leaf}_T(x),$$

where $G \parallel (b,a)$ appends progress record (b,a), and $T/(x,a)$ removes the processed call from the current leaf and records any selected choice. Write $\mathsf{HRes}((G,T),r)$ for iterating this update over a resolved word r.

$\mathsf{addr}(T)$ is the set of branch and group IDs that an edit may currently name. It includes each current group and the current part of a sequence.

K maps checkpoint IDs to immutable earlier branch records. Write $\mathcal{C}_k = \mathsf{contract}(K(k))$ for the remaining contract stored by checkpoint k. Σ_ζ is an immutable, versioned registry of typed edit rules. ρ records, for each workflow call, its action, authorization token, source region, scope, canonical tool request, and request digest. For each current workflow call x, $\beta(x) = (j,\eta)$ records the rule entry j and version η used to check it. χ records processed workflow calls in order, including whether each created or reused an authorization record, and ver is the state version. χ records authorization and workflow progress, while the remote service records completion and the returned value.

$\mathsf{Save}(k,b)$ creates a checkpoint for a current branch b under a fresh checkpoint ID k. A trusted transaction stores a protected copy of branch b's remaining contract $\mathcal{C}_b$, ordered progress record χ_b, relevant calls, and signed outcome requirements. The checkpoint check requires $\mathcal{C}_b = \Gamma_b/\mathsf{raw}(\chi_b)$, and the transaction advances ver without changing the current workflow, authorization records, β, the active rule version, or the authorization sequence. This internal transition changes H,

so a rule update computed from the earlier record will fail its final check. Restore accepts only records created by this rule.

2) Well-formedness:

Definition 1 (Well-formed workflow record)**.** *H is well formed when the following groups hold.*

1) Structure. *G is acyclic, and branch and group IDs are globally unique across the recorded DAG and current workflow. Every ID that an edit may name has a unique context decomposition, and every node in $\mathsf{heads}(T)$ is currently maximal. Every checkpoint names an immutable node and stores copies of its remaining contract and ordered progress record. Every original or edit-introduced outcome has exactly one signed policy authority, preserved when the controller carries, copies, or checkpoints it.*
2) Identity and progress. *Workflow-call IDs and the j identifiers in β are unique, while workflow calls that refer to the same action agree on the trusted controller, scope, canonical request, digest, and source region. Processed workflow calls are downward closed in their pomset, consistent with every choice status, and absent from the remaining workflow.*
3) Current workflow. $\mathrm{dom}(\beta) = X_{[\![T]\!]}$*, and β maps every current workflow call x to exactly one entry j_x under the domain's current rule version η. Every current leaf $b{:}\mathcal{C}$ equals its immutable registered contract Γ_b after removing the workflow calls recorded in χ_b; the caller cannot replace it with a smaller outcome set. Every constructor in T is defined, so no complete sequence in any subtree is a strict prefix of another.*
4) Edit rules and domain. *Every edge and introduced contract follows one immutable edit rule at version ζ. Every current, checkpoint, or rule-introduced workflow call, action, source region, and scope belongs to ω. Protected actions, authorization logs, and policies of distinct domains are disjoint, and an edit never crosses a domain.*

The policy domain has exactly one active rule version, namely the version recorded in H.

3) Authorization and completion: The remaining parts of the execution record are policy version κ, a regular prefix-closed authorization policy $\mathcal{A} \subseteq \mathcal{U}^*$ over label alphabet $\mathcal{U}$, authorization log Δ, and queue out of requests awaiting release. Authorization records can only be appended. A completion update advances an authorization record's status and stores the returned value. Each authorization record binds its action, immutable canonical tool request, creator workflow call, and authorization label. Each entry in χ points either to the authorization record it created or to the earlier record it reused, so (χ, Δ) determines retry and return lookup. $\mathsf{lab}(\Delta)$ is the sequence of authorization labels on calls that create authorization records. Restore may reconstruct workspace data but leaves Δ unchanged.

The execution record $(\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$ is *well formed* when H is well formed, $\mathsf{lab}(\Delta) \in \mathcal{A}$, each action has at most one authorization record, each processed call names the record it created or reused, the authorization log, request queue, and policy belong to ω, and the queue preserves the order of new authorizations. All definitions and results below quantify over such well-formed execution records.

C. Adversary and trusted boundary

The adversary controls Agent output, edit requests, tool arguments, copied workspace, retries, stale authorization tokens, and scheduling, including races between calls and edits. It cannot forge registries, the checkpoint directed acyclic graph (DAG), the record of which calls refer to the same action, the policy, or the authorization log. The policy authority signs policies, outcome requirements, and authorized removals. The trusted controller registers protected calls, actions, and authorization tokens, and it derives and checks each edit. The trusted controller checks the answer again before atomically changing the rules in force. The policy authority, trusted controller, protected registries and authorization log, proof verifier, and transaction mechanism form the trusted computing base (TCB). Within one policy domain, the TCB mediates every protected call and commits every state change and visible edit answer through a linearizable transaction. After an unexpected stop and restart, each transaction appears entirely before or entirely after its linearization point. Before the checker reads the execution record, the trusted controller may instantiate finitely many registered templates for tool calls. For tool request m, it computes canonical request $m^\circ = \mathsf{canon}_\kappa(m)$, reuses an action certified for m° or allocates a fresh one, and signs $(x, j, d, \mathsf{scope}, \mathsf{digest}(m^\circ), \mathsf{origin})$ into the authenticated registry. The resulting finite model $\mathcal{R}$ contains the workflow's protected calls and remains unchanged while the checker evaluates that execution record. Unmatched calls fail closed. The platform supplies workflow $\mathcal{W}$, finite model $\mathcal{R}$, edit rules, request-normalization rules, policy, checkpoints, authorization log, and queued requests in one protected execution record. From these inputs and an Agent request naming an edit and its objects, the controller derives the edited workflow, required outcomes, the authorization decision for each call, the safe execution set or rejection proof, and the rule entry and authorization token for each remaining call. Every derived object therefore comes from the execution record rather than Agent-supplied semantics.

Registration turns the protected-call behavior of a typed Agent workflow into the finite model $\mathcal{R}$. $\mathsf{CheckReg}$ verifies exact correspondence of outcomes, call order, actions, and first-time authorization records. The model stores these traces as outcome-indexed runs $\mathsf{BRuns}(\mathcal{R})$, together with a map λ from source traces. Write $\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$ when λ is a bijection preserving outcomes, which calls refer to the same action, call order, outcome authority, and authorization-log updates. Definition 6 and lemma 13 give the exact finite check and proof. Registered contracts enter through $\mathsf{CheckReg}$, and an idempotent or queryable remote service supplies authenticated completion records for its actions.

D. Security Objective

The security objective is fixed independently of the checker that will implement it. For an edit derived from the execution

record, a safe execution set satisfies four requirements. First, it preserves every source outcome that is neither already satisfied nor explicitly authorized for removal, and every admitted execution belongs to the derived target workflow. Second, every admitted prefix is allowed by the policy after accounting for authorization records already present in the append-only log. Third, each admitted prefix extends to a complete admitted execution. Fourth, every outcome still compatible with an admitted prefix retains such a completion. The last requirement rejects the example in section II, where separately safe outcomes cannot remain safe under one set of runtime rules. Definition 2 states these requirements over executions labeled by outcome. Lemma 4 then proves that the checker computes the largest safe execution set.

IV. Exact Checking of Execution Edits

The checker first derives the exact edited workflow from the execution record and the requested edit. It then decides whether each workflow call creates or reuses an authorization record and removes precisely those complete executions that violate the security objective after some prefix.

A. Constructing the Edited Workflow

1) Checking an edit rule: The six disjoint constructors of Edit_6 are exactly the typed operation signatures in table I. Save, the initial rule update, later rule registration, and protected calls use their own transitions.

An Agent request u names an edit kind, current object IDs, and edit-rule ID σ. Registry Σ_ζ records the expected source and checkpoint patterns for σ, any workflow steps the edit adds, the source region for each copy, and the authority that signs each introduced outcome.

The checker constructs explicit preservation maps for every region that the rule carries or copies. For source contract $\mathcal{C}$, target region $\mathcal{D}$, and parent map $\mu : X_\mathcal{D} \to X_\mathcal{C}$, write $\mathcal{C} \preccurlyeq_\mu \mathcal{D}$ exactly when μ is a global bijection, some surjection $\pi : O_\mathcal{D} \twoheadrightarrow O_\mathcal{C}$ satisfies the conditions below for every target outcome v, and μ preserves actions and source regions.

$$x \in X_{p_v} \iff \mu(x) \in X_{p_{\pi(v)}},$$
$$x \prec_v y \iff \mu(x) \prec_{\pi(v)} \mu(y).$$

Global bijection and workflow call–outcome membership equivalence prevent splitting a source workflow call shared across outcomes into outcome-specific target workflow calls.

For candidate T', let R_u index the disjoint regions carried or copied inside the replaced subtree. $\mathcal{P}_u = \{(\mathcal{C}_r, \mathcal{D}_r, \mu_r, \pi_r)\}_{r \in R_u}$ collects these preservation maps. A separate identity map verifies that the surrounding workflow preserves syntax, outcomes, workflow calls, actions, source regions, and order. Writing X_{ctx} for its workflow calls and $X_{\mathrm{added(u)}}$ for workflow calls added by the rule, the checker verifies the following disjoint cover.

$$X_{[\![T']\!]} = X_{\mathrm{ctx}} \uplus \biguplus_{r \in R_u} X_{\mathcal{D}_r} \uplus X_{\mathrm{added(u)}}.$$

The edit rule and surrounding workflow induce $\mathsf{pr}^u_r : O_{[\![T']\!]} \rightharpoonup O_{\mathcal{D}_r}$. For choice, this projection is defined only in the arm containing r, while product and sequence select the corresponding indexed component. Define

$$\mathsf{EditedRequired}_u = \{(r, o) \mid r \in R_u,\ o \in O_{\mathcal{C}_r}\},$$
$$\mathsf{Covers}_u(t, r, o) \iff \exists v \in O_{\mathcal{D}_r}.\quad \mathsf{pr}^u_r(t) = v \ \wedge\ \pi_r(v) = o.$$

Let $O_{\mathrm{unchanged}}$ be the outcomes outside the edited region, and let $\iota^u_{\mathrm{ctx}} : O_{\mathrm{unchanged}} \hookrightarrow O_{[\![T']\!]}$ be their unchanged embedding into the target. Define the outcomes that remain required and the target outcomes that cover them by

$$\begin{aligned} \mathsf{StillRequired}_u &= \mathsf{EditedRequired}_u \\ &\quad \uplus (\{\mathsf{ctx}\} \times O_{\mathrm{unchanged}}), \\ \mathsf{Covers}_u(t, (\mathsf{ctx}, o)) &\iff \\ &\quad t = \iota^u_{\mathrm{ctx}}(o). \end{aligned} \tag{4}$$

For $a = (r, o) \in \mathsf{EditedRequired}_u$, $\mathsf{Covers}_u(t, a)$ denotes the relation above. The checker also requires $\forall a \in \mathsf{StillRequired}_u.\ \exists t \in O_{[\![T']\!]}.\ \mathsf{Covers}_u(t, a)$: every required source outcome has a complete target outcome, shared workflow calls remain shared, and inherited calls remain distinguished from steps added by the edit rule.

The edit-rule check uses the following judgment.

$$\Sigma_\zeta; H \vdash_{\mathrm{edit}} u \Rightarrow (T', \mathcal{P}_u)$$

It holds exactly when every named object belongs to $\mathsf{addr}(T)$, every introduced object belongs to ω, the source pattern matches, the applicable preservation maps and disjoint cover hold, every still-required outcome is covered, every removal carries the proper authority, and $[\![T']\!]$ is defined. The policy authority may authorize removal of the unselected arm of a typed MergeSelect or the current branch of RestoreReplace, using the signed sets defined below.

2) Required-outcome preservation: The registered edit rule determines every outcome that the edit must account for. Fork accounts for both copies, Restore for the current and checkpoint branches, MergeSelect for the selected and unselected arms, and MergeJoin for both arms. Outcomes outside the edited region remain unchanged. Definition 4 defines the outcomes required before the edit, those already satisfied, and those whose removal the policy authority signed. The checker accepts the edit rule when the target-derived set from equation (4) satisfies

$$\begin{aligned} \mathsf{StillRequired}_u = \mathsf{RequiredBefore}(H, u) \\ \setminus \big(\mathsf{Satisfied}_{H,u} \uplus \mathsf{AuthorizedRemoval}_{H,u}\big), \end{aligned}$$

and hence the exact disjoint partition

$$\begin{aligned} \mathsf{RequiredBefore}(H, u) = \mathsf{StillRequired}_u \\ \uplus\, \mathsf{Satisfied}_{H,u} \\ \uplus\, \mathsf{AuthorizedRemoval}_{H,u}. \end{aligned}$$

This partition accounts for every source outcome and prevents an Agent request from silently dropping one. The preservation maps retain actions, source regions, and order, while the rule update retains the records for satisfied outcomes and authorized removals.

Let $\mathcal{E}[-]$ be the current workflow with one editable region replaced by a hole. carry preserves branch, outcome, and

workflow-call IDs, pomset order, actions, signed outcome authorities, base contracts, ordered progress records, and nested choices. clone_r assigns fresh branch, outcome, and workflow-call IDs while preserving pomset order, actions, scope, digest, source regions, and the authority for each outcome. Each copied branch starts from its registered remaining contract with an empty progress record. Branches added by an edit rule likewise start from their registered contract and an empty progress record. The contracts $E^L_\sigma, E^R_\sigma, E^J_\sigma$ are additional workflow steps derived from the rule registry. Define

$$L_\sigma = \mathsf{clone}_L(\mathcal{C}) \triangleright E^L_\sigma, \qquad R_\sigma = \mathsf{clone}_R(\mathcal{C}) \triangleright E^R_\sigma.$$

New identifiers and the target rule version η^+ are deterministically allocated from $(\omega, \zeta, \mathsf{ver}, u, \mathsf{role})$. role identifies which operation argument receives each new ID. Write $\mathsf{Copy}_r(\mathcal{C})$ for the proof tuple $(\mathcal{C}, \mathsf{clone}_r(\mathcal{C}), \mu_r, \pi_r)$, and $\mathsf{Keep}(\mathcal{C})$ for $(\mathcal{C}, \mathsf{carry}(\mathcal{C}), \mu, \pi)$. Each abbreviation asserts the corresponding $\preccurlyeq_\mu$ relation. For a current workflow T_0, $\mathsf{Keep}(T_0)$ abbreviates $\mathsf{Keep}([\![T_0]\!])$ and has target contract $[\![\mathsf{carry}(T_0)]\!]$. Every b or g named below must lie in $\mathsf{addr}(T)$.

In an open choice, the first processed workflow call atomically selects its arm and disables later calls from the other arm, whether that first call creates or reuses an authorization record. Consequently, $\mathsf{MergeSelect}(g, w, \sigma)$ is valid precisely while the unselected arm has made no recorded progress. MergeJoin replaces the current $\|_g$ node with join, removes g from the set of IDs an edit may name, and prevents the same pair from being joined again.

Every edit replaces all runtime rules for its policy domain, and $\beta(x)$ uses $\eta^- = \eta$ for every current call x.

3) Preparing calls for the new rules: The execution-edit judgment consists exactly of one row of table I applied inside the well-typed workflow context $\mathcal{E}$, followed by the state update below. Let e_u contain the edit record and exactly the copied, added, and group nodes prescribed by T'; carried node IDs remain unchanged. For every current target workflow call x, the trusted controller deterministically creates a rule entry j^+_x and authorization token h^+_x. The updated record ρ' preserves the action, source region, scope, digest, and retry information. Define

$$\mathcal{H}^+_u = \{(x, j^+_x, h^+_x) \mid x \in X_{[\![T']\!]}\},$$
$$\beta'(x) = (j^+_x, \eta^+).$$

The new authorization tokens are inactive and inaccessible to the caller before the rule update takes effect. The complete judgment is shown below.

$$\begin{aligned} &\Sigma_\zeta; H \vdash u \Downarrow (H', \mathcal{C}_{H,u}, \eta^-, \eta^+), \\ &\qquad H' = (\omega, G \cdot e_u, T', K, \Sigma_\zeta, \rho', \beta', \chi, \mathsf{ver}+1, \eta^+), \\ &\qquad \mathcal{C}_{H,u} = [\![T']\!], \qquad \eta^- = \eta. \end{aligned} \tag{5}$$

Here $G \cdot e_u$ retains earlier progress, outcome-authority records, and existing nodes, and initializes each copied or added branch from its remaining registered contract with inherited or rule-signed outcome authority and no progress. Thus an edit preserves K, Σ_ζ, χ, appends one record of the edit and its target nodes, creates a new rule entry and authorization token for every current call, and moves the domain to η^+. Table constructors determine new group modes, while carried nested modes remain unchanged. A nonexistent checkpoint, wrong group mode, nonmember winner, mismatched rule pattern, unauthorized removal, or undefined composition produces Invalid. Independent domains step by asynchronous product, and each domain replaces all of its runtime rules at once.

Lemma 2 (Exact edit derivation and atomic rule update)**.** *For well-formed H, the judgment in equation* (5) *is deterministic. If it derives H', then H' is well formed. It preserves required outcomes and their signed authorities, retaining evidence for every satisfied outcome and every still-required source outcome except a signed authorized removal. The surrounding-workflow identity map and family $\mathcal{P}_u$ preserve which calls refer to the same action, source regions, the authority for each outcome, and causal order. For every $a \in \mathsf{StillRequired}_u$, some complete target outcome t satisfies $\mathsf{Covers}_u(t, a)$, including every unchanged outcome outside the edited region. Every current target workflow call x has $\beta'(x) = (j^+_x, \eta^+)$ and an authorization token for j^+_x.*

Proof sketch. The six cases use the copy/keep bijections and surrounding-workflow identity map to preserve action sharing, source regions, and order and to construct every Covers_u witness. The common state update establishes well-formedness and sets $\beta'(x) = (j^+_x, \eta^+)$ for every current target workflow call. Section A-D1 gives the full argument. □

Recall the authorization-label alphabet $\mathcal{U}$ and regular prefix-closed policy $\mathcal{A} \subseteq \mathcal{U}^*$. For authorization log Δ, $\mathsf{lab}(\Delta) \in \mathcal{A}$ is its authorization sequence and D_Δ is the set of actions that already have authorization records. Fix $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$ and write $\mathsf{state}(\Theta) = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$. The initial rule update, each execution edit, and each later registered extension produce a unique derived record. Write $\mathsf{Derive}(\Theta) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. Root derivation additionally requires that $\mathsf{CheckReg}(\mathcal{W}, \mathcal{R})$ accept. An execution edit has $(\kappa_u, \mathcal{A}_u) = (\kappa, \mathcal{A})$ and uses the unique execution-edit judgment. $\mathsf{Extend}(\xi)$ adds authenticated edit-rule, call, outcome-authority, and policy entries while preserving the current workflow, its progress, and all earlier authorizations; section A-D2 gives the judgment. Let ρ_u be the target registry in H_u, let $O_{H,u} = O_{\mathcal{C}_u}$, and use $H' = H_u$ and $\rho' = \rho_u$ where earlier notation is convenient. The execution sets below use Θ, the edit-rule registry, ρ_u, and target policy $\mathcal{A}_u$. Before the rules change, the trusted controller checks the complete record again, including queued requests out, so a concurrent release makes an earlier answer stale. We abbreviate the fixed parameters by writing H, u.

B. New and Reused Authorizations

Function $\mathsf{Resolve}^{\rho_u}_\Delta$ keeps every workflow call and annotates it as creating a new authorization or reusing an earlier one.

TABLE I
THE SIX AGENT EXECUTION EDITS. $E^{\bullet}_{\sigma}$ IS DERIVED FROM THE REGISTERED EDIT RULE IN Σ_{ζ}.

Operation u	Required current shape	Derived target shape	Checked preservation
$\mathsf{ForkChoice}(b,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}]$	$\mathcal{E}[b_L{:}L_\sigma \oplus_g^{\mathsf{open}} b_R{:}R_\sigma]$	$\{\mathsf{Copy}_L(\mathcal{C}), \mathsf{Copy}_R(\mathcal{C})\}$
$\mathsf{ForkParallel}(b,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}]$	$\mathcal{E}[b_L{:}L_\sigma \parallel_g b_R{:}R_\sigma]$	$\{\mathsf{Copy}_L(\mathcal{C}), \mathsf{Copy}_R(\mathcal{C})\}$
$\mathsf{RestoreReplace}(b,k,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}],\ k \in \operatorname{dom} K$	$\mathcal{E}[b_k{:}\mathsf{clone}_k(\mathcal{C}_k)]$	$\{\mathsf{Copy}_k(\mathcal{C}_k)\}$; current branch removed
$\mathsf{RestoreLive}(b,k,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}],\ k \in \operatorname{dom} K$	$\mathcal{E}[b{:}\mathsf{carry}(\mathcal{C}) \parallel_g b_k{:}\mathsf{clone}_k(\mathcal{C}_k)]$	$\{\mathsf{Keep}(\mathcal{C}), \mathsf{Copy}_k(\mathcal{C}_k)\}$
$\mathsf{MergeSelect}(g,w,\sigma)$	$\mathcal{E}[T_L \oplus_g^s T_R],\ s \in \{\mathsf{open}, w\}$	$\mathcal{E}[\mathsf{carry}(T_w); b_J{:}E^J_\sigma]$	$\{\mathsf{Keep}(T_w)\}$; other arm removed
$\mathsf{MergeJoin}(g,\sigma)$	$\mathcal{E}[T_L \parallel_g T_R]$	$\mathcal{E}[\mathsf{join}(\mathsf{carry}(T_L), \mathsf{carry}(T_R)); b_J{:}E^J_\sigma]$	$\{\mathsf{Keep}(T_L), \mathsf{Keep}(T_R)\}$

Starting with $D_0 = D_\Delta$, it maps a call sequence $w = x_1 \cdots x_n$ to annotated sequence $z_1 \cdots z_n$. For $d_i = q^{\rho_u}_{\mathrm{act}}(x_i)$,

$$z_i = \begin{cases} \mathsf{new}(x_i, d_i), & d_i \notin D_{i-1}, \quad D_i = D_{i-1} \cup \{d_i\}; \\ \mathsf{reuse}(x_i, d_i), & d_i \in D_{i-1}, \quad D_i = D_{i-1}. \end{cases} \tag{6}$$

auth keeps the authorization label of each new authorization and contributes nothing for reuse.

$$\mathsf{auth}(\mathsf{new}(x,d)) = \ell_{\rho_u}(d), \qquad \mathsf{auth}(\mathsf{reuse}(x,d)) = \epsilon.$$

For annotated z, $\mathsf{raw}(z)$ drops the new/reuse annotation and action while preserving the ordered workflow-call IDs. A Retry repeats the same canonical request and adds no workflow call. Restored workflow calls that share an action remain distinct. The first call creates the authorization record, and later calls reuse it.

Lemma 3 (New/reuse classification)**.** $\mathsf{Resolve}^{\rho_u}_\Delta$ *is deterministic, preserves length, and gives the same annotations on every shared prefix. Among calls annotated as new, each action occurs at most once outside D_Δ, and dropping the annotations gives exactly the input sequence. Moreover, if* $\mathsf{Resolve}^{\rho_u}_\Delta(rv) = zv'$*, then* $z = \mathsf{Resolve}^{\rho_u}_\Delta(r)$ *and* $v' = \mathsf{Resolve}^{\rho_u}_{\Delta_z}(v)$*, where Δ_z extends Δ by the new authorizations in z.*

Proof. Induction on input length shows that the function emits one prefix-determined annotation per input symbol and updates D_i exactly when it creates a new authorization. Set membership ensures at most one new authorization per action. Both cases retain x_i. Splitting after r gives the stated composition equation. □

Thus a retry changes neither the processed-call record nor the authorization sequence, although its successful API reply is observable as $\mathsf{Return}(t, [v]_{\cong})$. A copied workflow call that reuses an authorization record advances the processed-call record without changing the authorization sequence.

C. Safe Completion After Every Prefix

1) Computing all safe behavior: For each derived outcome $o \in O_{H,u}$, define all of its complete executions and the subset allowed by policy.

$$\begin{aligned} R_o(H,u) &= \{\mathsf{Resolve}^{\rho_u}_\Delta(w) \mid w \in \mathsf{Lin}(p_o)\}, \\ M_o(H,u) &= \{z \in R_o(H,u) \mid \mathsf{lab}(\Delta)\mathsf{auth}(z) \in \mathcal{A}_u\}. \end{aligned} \tag{7}$$

For an annotated prefix z, let

$$\mathsf{Compat}(z) = \{o \mid \mathsf{raw}(z) \preceq_{\mathrm{pref}} w \text{ for some } w \in \mathsf{Lin}(p_o)\}.$$

$\mathsf{PreservesRequired}(H, u, \widehat{B})$ means that every outcome in $\mathsf{StillRequired}_u$ is covered by some target outcome represented in $\widehat{B}$:

$$\forall a \in \mathsf{StillRequired}_u. \quad \exists (t,y) \in \widehat{B}.\ \mathsf{Covers}_u(t,a).$$

Definition 2 (Safe execution set)**.** *A safe execution set is a pair $(L, \widehat{B})$, where L contains allowed partial executions and $\widehat{B}$ contains allowed complete executions labeled by outcome, satisfying the conditions below.*

1) $\widehat{B} \neq \varnothing$*;*
2) workflow and outcomes: $\mathsf{PreservesRequired}(H, u, \widehat{B})$*,* $\widehat{B} \subseteq \biguplus_{o \in O_{H,u}} \{o\} \times R_o(H,u)$*, and* $L \subseteq \mathsf{Pref}(\bigcup_{o \in O_{H,u}} R_o(H,u))$*;*
3) policy: $\mathsf{lab}(\Delta)\,\mathsf{auth}(z) \in \mathcal{A}_u$ *for every* $z \in L$*;*
4) some completion after every prefix: $L = \mathsf{Pref}(\pi_2 \widehat{B})$*; and*
5) every compatible outcome remains completable: *for every* $z \in L$ *and* $o \in \mathsf{Compat}(z)$*, some* $(o,y) \in \widehat{B}$ *extends* z*.*

Checking each outcome separately is insufficient. After a partial execution shared by several outcomes, every outcome still compatible with that partial execution must retain a policy-safe completion. The operator below removes exactly the complete executions that violate this condition. Put

$$\widehat{B}_0 = \biguplus_{o \in O_{H,u}} \{o\} \times M_o.$$

For $\widehat{B} \subseteq \widehat{B}_0$, define the monotone operator

$$\widehat{\Phi}(\widehat{B}) = \left\{ (o,y) \in \widehat{B}_0 \;\middle|\; \begin{array}{l} \forall z \preceq_{\mathrm{pref}} y.\ \forall o' \in \mathsf{Compat}(z). \\ \exists (o',y') \in \widehat{B}.\ z \preceq_{\mathrm{pref}} y' \end{array} \right\}. \tag{8}$$

Set $\widehat{B}_{i+1} = \widehat{\Phi}(\widehat{B}_i)$. Since $\widehat{B}_1 \subseteq \widehat{B}_0$, repeated application removes executions until the finite set stops changing:

$$\widehat{B}^{\dagger}_{H,u} = \nu \widehat{B}.\widehat{\Phi}(\widehat{B}) = \bigcap_{i \geq 0} \widehat{B}_i,$$

$$B^{\dagger}_{H,u} = \pi_2 \widehat{B}^{\dagger}_{H,u}, \qquad W^{\dagger}_{H,u} = \mathsf{Pref}(B^{\dagger}_{H,u}).$$

Definition 3 (Safe execution edit)**.**

$$\mathsf{SafeEdit}(\Theta) \iff \begin{cases} \widehat{B}^{\dagger}_{H,u} \neq \varnothing, & \mathsf{Derive}(\Theta) \textit{ is defined}; \\ \mathsf{false}, & \textit{otherwise}. \end{cases} \tag{9}$$

At $z = \epsilon$, nonemptiness covers every required outcome. The existential quantifier chooses a call order, and the universal condition keeps every compatible outcome completable after every prefix.

Lemma 4 (Largest safe execution set)**.** *A safe execution set exists iff* $\mathsf{SafeEdit}(\Theta)$*. When the edit is safe,* $(W^{\dagger}_{H,u}, \widehat{B}^{\dagger}_{H,u})$ *is a safe execution set, and every safe execution set* $(L, \widehat{B})$ *satisfies* $\widehat{B} \subseteq \widehat{B}^{\dagger}_{H,u}$ *and* $L \subseteq W^{\dagger}_{H,u}$*.*

Proof. The workflow, outcome, and policy conditions place every safe execution set's complete executions inside $\widehat{B}_0$. The final condition makes this set post-fixed, so monotonicity and finite Knaster–Tarski place it inside $\widehat{B}^{\dagger}$, and projected prefixes give $L \subseteq W^{\dagger}$. Conversely, $\widehat{B}^{\dagger} = \widehat{\Phi}(\widehat{B}^{\dagger})$ keeps every compatible outcome completable. At the empty prefix, this condition and the checked coverage witnesses from lemma 2 establish $\mathsf{PreservesRequired}$. The inclusion $\widehat{B}^{\dagger} \subseteq \widehat{B}_0$ ensures that complete executions follow the edited workflow and policy, while prefix closure of $\mathcal{A}_u$ makes every allowed prefix safe. Finally, $W^{\dagger} = \mathsf{Pref}(\pi_2 \widehat{B}^{\dagger})$ gives a completion after every allowed prefix. Hence the fixed point is a safe execution set exactly when nonempty. □

$\mathsf{MaxSafe}(\Theta; L, \widehat{B})$ means that $(L, \widehat{B})$ is safe and contains every other safe execution set. When such a pair exists, write its unique value as $\mathsf{SafeBehavior}(\Theta)$. Lemma 4 proves that this witness exists exactly when $\widehat{B}^{\dagger}_{H,u} \neq \varnothing$ and then equals $(W^{\dagger}_{H,u}, \widehat{B}^{\dagger}_{H,u})$.

2) Why Outcome-by-Outcome Checking Fails: Section II introduced the contract $X \otimes (Y \oplus Z)$ under policy $\mathsf{Pref}\{yx, xz\}$. For its annotated calls, write $\bar{x} = \mathsf{new}(x_o, d_x)$, $\bar{y} = \mathsf{new}(y_o, d_y)$, and $\bar{z} = \mathsf{new}(z_o, d_z)$. The candidate family initially contains $\bar{y}\bar{x}$ and $\bar{x}\bar{z}$. The first fixed-point iteration removes $\bar{x}\bar{z}$. The second iteration removes $\bar{y}\bar{x}$, because after the first removal the empty prefix no longer has a completion for the $X \otimes Z$ outcome. The empty fixed point is the formal reason the exact checker rejects.

Proposition 1 (Exact correspondence with generalized nonblocking)**.** *For nonempty* $\widehat{B} \subseteq \widehat{B}_0$*, let* $\mathcal{T}_{\widehat{B}}$ *be the prefix trie of* $L_{\widehat{B}} = \mathsf{Pref}(\pi_2 \widehat{B})$*. Mark state* z *by* $\mathsf{possible}_o$ *exactly when* $o \in \mathsf{Compat}(z)$*, and mark state* y *by* done_o *exactly when* $(o, y) \in \widehat{B}$*. Then*

$$\widehat{B} \subseteq \widehat{\Phi}(\widehat{B}) \iff \forall o \in O_{H,u}.\ \mathcal{T}_{\widehat{B}} \text{ is } (\mathsf{possible}_o, \mathsf{done}_o)\text{-nonblocking}.$$

Consequently, if $\widehat{B}^{\dagger}$ *is nonempty, it is the largest nonempty completion subfamily of* $\widehat{B}_0$ *satisfying these outcome-specific nonblocking conditions. If it is empty, no nonempty subfamily satisfies the conditions. Ordinary marker nonblocking of the unaugmented projected language is strictly weaker.*

Proof. From reachable prefix z, a done_o-marked state is reachable exactly when some $(o, y) \in \widehat{B}$ extends z. Quantifying over exactly the states marked $\mathsf{possible}_o$ is therefore the post-fixed-point condition. Greatestness follows from the greatest-fixed-point construction. For strictness, the preceding instance has the marker-nonblocking projected pair $(\mathsf{Pref}\{\bar{y}\bar{x}, \bar{x}\bar{z}\}, \{\bar{y}\bar{x}, \bar{x}\bar{z}\})$, but its indexed fixed point is empty. □

Proposition 1 relates the fixed point to generalized nonblocking after the controller has derived the outcome-specific completion conditions [16], [17]. The controller derives those conditions from the requested edit, authorized removals, actions, authorization records, and the execution record. It also decides whether each protected call creates or reuses an authorization, computes every safe continuation or a rejection proof, and puts an accepted result into effect atomically.

3) Rejection Proofs: Each removed (o_{rem}, y) records its rank i, a prefix $z \preceq y$, and uncovered $o_{\mathrm{miss}} \in \mathsf{Compat}(z)$. Induction shows that every post-fixed P is contained in every $\widehat{B}_i$. An empty final family therefore proves that no safe execution set exists. Before the rule update, the trusted controller recomputes both the removal order and the remaining completions.

4) Checking Again After a Permitted Prefix: To show that the checker can be applied again after each permitted partial execution, fix $H, u, \mathcal{A}_u$ after deriving $\mathcal{C}$, and write $W^{\dagger}(\mathcal{C}, \Delta)$ and $\widehat{B}^{\dagger}(\mathcal{C}, \Delta)$ with those parameters suppressed.

Lemma 5 (Rechecking after a prefix)**.** *If* $z \in W^{\dagger}(\mathcal{C}, \Delta)$*, then* $\mathsf{Resolve}^{\rho'}_{\Delta}(\mathsf{raw}(z)) = z$*,* $\mathsf{lab}(\Delta_z) = \mathsf{lab}(\Delta)\mathsf{auth}(z)$*, and*

$$W^{\dagger}(\mathcal{C}, \Delta)/z = W^{\dagger}(\mathcal{C}/\mathsf{raw}(z), \Delta_z),$$
$$\widehat{B}^{\dagger}(\mathcal{C}, \Delta)/z = \widehat{B}^{\dagger}(\mathcal{C}/\mathsf{raw}(z), \Delta_z),$$

where $\widehat{B}/z = \{(o, v) \mid (o, zv) \in \widehat{B}\}$*.*

Proof sketch. Streaming resolution recovers the original calls, gives the authorization-log equality, and makes the candidate executions after z identical on both sides. Removing prefix z from $\widehat{B}^{\dagger}$ gives a post-fixed family for the next check, while adding z to any such family gives one for the original check. Greatestness yields the fixed-point equality, and $\mathsf{Pref}(B)/z = \mathsf{Pref}(B/z)$ yields the generated-language equality. The full proof is in section A-D3. □

Thus every accepted prefix retains a safe completion for every outcome that remains compatible with that prefix.

5) Checker Outputs: The checker returns $\mathsf{Invalid}$ when derivation fails, Reject with the ordered reasons for removing every execution when the fixed point is empty, and $\mathsf{Installable}$ with the largest safe execution set otherwise. Write $\mathsf{Compile}(\Theta)$ for this deterministic output. Each answer includes a digest of the execution record that was checked. Section A-C gives the exact returned data and verification rules. Proposition 2 proves an output-sensitive $O(D + n\Lambda + |\mathcal{G}_{\mathrm{cov}}| + V_B \log(2 + V_B))$-time, $O(D + \Lambda + |\mathcal{G}_{\mathrm{cov}}|)$-space bound. Exponential growth occurs only in explicitly emitted linearizations and prefix–required-outcome support pairs.

V. Runtime Enforcement

The safety checker produces either a rejection proof or the largest safe family of complete executions. The trusted controller turns a nonempty family into rules checked before each tool call. It builds a finite automaton for the allowed

partial executions, records which version of the execution record was checked, and atomically puts the new rules into effect. The protocol orders every overlapping protected call either before or after the atomic rule update. The proof compares this implementation with an ideal machine that puts an accepted result into effect in one atomic step and admits exactly its resolved prefixes. The ideal transition system and the field-level implementation rules appear in section A-H and table III. This section gives the automaton construction, the invariant after the rules change, and the atomic update used by the main theorem.

A. Turning Safe Executions into Runtime Rules

Removing outcome labels from the largest safe execution set gives the finite pair $(W, B) = (W^{\dagger}_{H,u}, B^{\dagger}_{H,u})$. For $L/z = \{v \mid zv \in L\}$, let $q_z = (W/z, B/z)$ be the automaton state after prefix z.

$$\begin{aligned} Q_{W,B} &= \{q_z \mid z \in W\}, \\ q^0 &= q_\epsilon = (W, B), \\ q_z \xrightarrow{a} q_{za} \quad &\Longleftrightarrow \quad za \in W. \end{aligned} \tag{10}$$

Call this deterministic automaton $\mathsf{Can}(W, B)$. Every state is reachable and is marked exactly when $\epsilon \in B^{\dagger}/z$. Each transition symbol a contains workflow call x, action d, and its new or reuse mode, so one workflow call cannot use another call's rule entry.

Lemma 6 (Exact finite enforcement)**.** *The generated and marked languages of the automaton in equation* (10) *are exactly* $(W^{\dagger}_{H,u}, B^{\dagger}_{H,u})$. *The automaton is the smallest deterministic marked partial automaton for this pair up to state renaming.*

Proof. Induction on z shows that the automaton reaches $(W^{\dagger}/z, B^{\dagger}/z)$, enables a iff $za \in W^{\dagger}$, and marks the reached state iff $z \in B^{\dagger}$. Distinct states differ on a generated or marked suffix and must remain distinct. The automaton is the paired-language derivative construction [20]. □

B. Atomic Rule Update

Before the rules change, the trusted controller re-derives the edit and recomputes the fixed point from the execution record. If any protected call has committed since the candidate was computed, the candidate is stale and changes nothing. Otherwise, one policy-domain transaction closes the old rule version, activates the new automaton, replaces the rule entry and authorization token for every current workflow call, and only then exposes the successful edit answer. Protected calls verify their workflow call, action, canonical request, authorization token, and enabled automaton transition in the same transaction that records progress. A new call creates one authorization record and queues the canonical request. An reuse call advances the workflow by linking to the existing record without extending the authorization sequence.

a) Invariant after the rule update: We write $\mathsf{AgentSec}(S; c, r)$ when five conditions hold: (1) the workflow now in force and its required outcomes come from the checked edit; (2) the stored nonempty execution family is the checker's largest safe family; (3) the workflow, authorization log, queue, and checkpoints match the executed prefix r; (4) the automaton is in exactly the state reached after r; and (5) exactly one rule version is active, with one rule entry and authorization token for every remaining workflow call. For a submitted tool request, one transaction verifies the workflow call, action, canonical request, authorization token, active rule version, and enabled automaton transition before recording any progress. The same transaction either creates the authorization record and queues the request, or records reuse of an existing authorization record and its return value.

b) Final check: For current state S, write $\Theta_S(u) = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$ for the safety-check instance obtained from its current components. Write $\mathsf{Ready}(S, u, Y)$ when the service re-derives u from the current execution record, recomputes the same nonempty fixed point and automaton carried by Y, confirms that the record has not changed since Y was computed, confirms that the old rule version is current, and confirms that the new version is inactive. If these checks pass, one transaction closes the old version, activates the new automaton, replaces every current rule entry and authorization token, and then exposes the successful edit answer. A changed state makes the candidate stale and forces a new check. The complete state updates, retries, request release, completion updates, and recovery rules appear in section A-I and table III.

Lemma 7 (Edit–call serialization)**.** *In every reachable well-formed runtime state, a visible edit answer and a protected call in the same policy domain have a unique order at the atomic rule update. If the call commits first, it changes the execution record, so the earlier result cannot take effect. If* Install *commits first, the old call cannot resolve* new *or* reuse*. Steps in independent domains commute by product semantics.*

Proof. If the call commits first, its recorded progress changes the execution record. If the rule update commits first, it closes the old rule version, so the old authorization token fails. Atomicity excludes a partial third outcome. Independent domains commute. □

C. Trust Boundary

The TCB is the trusted boundary defined in section III. Search, minimization, and candidate preparation remain untrusted because the final check recomputes them. The next section states the exactness and runtime guarantees obtained at this boundary.

VI. Formal Guarantees

The theorem connects derivation from the execution record, exact checking, finite enforcement, the atomic rule update, and every later runtime step. Its central clause says that the checker accepts if and only if a safe runtime implementation exists. The other clauses establish faithful registration, the information required for an exact answer, repeated use, and runtime correctness.

When defined, write $\delta = \mathsf{Derive}(\Theta) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. A *safe runtime implementation* is

an automaton M together with complete executions $\widehat{B}_M$ labeled by outcome. The partial and complete executions of M must form a safe execution set as defined in definition 2. The atomic rule update activates this automaton at η^+, closes η^-, and replaces the rule entries and authorization tokens for all target calls.

a) What the checker must know: Write $V = \mathsf{Sec}(S; c, r) = \mathbf{P} \bowtie \mathbf{I} \bowtie \mathbf{E} \bowtie \mathbf{C}$ for four parts of the checked record: (1) the workflow and its required outcomes, (2) which calls refer to the same action, (3) earlier authorizations and call progress, and (4) which rules govern a call that overlaps the atomic rule update. Suppose a checker receives only some function $f(V)$ of this record. The function is exact if any two records that it makes indistinguishable always give the same answer to every common decision or rule-update request. Internal identifiers may be renamed consistently because their spelling cannot affect the answer. For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, π_J gives the checker exactly the parts named by J. Section A-F gives four pairs of records showing that each part of V is necessary.

b) Required-outcome preservation:

$$\mathsf{PreservesRequired}(H, u, \widehat{B}) \iff \forall a \in \mathsf{StillRequired}_u.\ \exists (t, y) \in \widehat{B}.\ \mathsf{Covers}_u(t, a).$$

Thus every required source outcome appears in an accepted target outcome.

c) Visible runtime behavior: $\mathcal{O}_{\mathrm{api}}$ includes edit answers tied to the checked record, edits put into effect, protected-call answers, released requests, completion updates, and returned values. It hides internal computation, Save, stale candidates, and restart bookkeeping. $\mathcal{B}$ matches ideal and runtime states on AgentSec and these visible fields. Section A-L gives the full relation.

Theorem 1 (Faithful Registration)**.** *For finite-state $\mathcal{W}$ and finite $\mathcal{R}$,* $\mathsf{CheckReg}(\mathcal{W}, \mathcal{R})$ *accepts iff stored λ witnesses $\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$. On acceptance, the source workflow's protected-call traces and* $\mathsf{BRuns}(\mathcal{R})$ *are order-isomorphic and preserve outcomes, actions,* new/reuse*, the safety answer, and the largest safe execution set. Malformed registrations fail before startup.*

For the remaining results, assume accepted registration, trusted checking of every protected call, and linearizable transactions that remain atomic after a restart within each policy domain. They hold for every $u \in \mathsf{Edit}_6 \uplus \{\mathsf{Extend}(\xi)\}$ at every finite reachable AgentSec state.

Theorem 2 (Exact Edit-Safety)**.** *The ideal and runtime machines return the same mutually exclusive and exhaustive answer for $\Theta = \Theta_S(u)$. Undefined derivation gives state-preserving* Invalid*. For defined $\delta = \mathsf{Derive}(\Theta)$, $\widehat{B}^{\dagger}_{H,u} = \varnothing$ gives state-preserving* Reject *with no safe runtime implementation. Otherwise, for $Y = \mathsf{Compile}(\Theta)$, nonempty $\widehat{B}^{\dagger}_{H,u}$, existence of a safe runtime implementation,* $\mathsf{Ready}(S, u, Y)$*, and a successor that puts the edit into effect while preserving* AgentSec *are equivalent. The largest execution family satisfies* PreservesRequired*, records every already-satisfied source outcome, and records every authorized removal.*

Theorem 3 (Necessary State Information)**.** *The full record is sufficient. If any one of the four parts is omitted, two records become indistinguishable even though the correct answer differs.*

$$\mathsf{Exact}(\pi_J) \iff J = \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

The four pairs in table II witness these opposite answers. In particular, a checker is not exact if it forgets which calls or authorization records refer to the same action, or if it sees only the target workflow proposed by the caller. Generalized nonblocking can serve as an exact solver only after the trusted controller derives the workflow and outcome-specific completion conditions.

Theorem 4 (Repeated-Use Safety)**.** *Every enabled event preserves* AgentSec*, so the theorem applies after every finite reachable sequence of edits, registered extensions, protected calls, authorization updates, rule updates, stops, and restarts.*

Theorem 5 (Runtime Correctness)**.** *$\mathcal{B}$ is a divergence-insensitive weak bisimulation under* $\mathsf{obs}_{\mathrm{api}}$*, so every pair related by $\mathcal{B}$ satisfies $S_0 \approx^{\mathsf{di}}_{\mathcal{O}_{\mathrm{api}}} I_0$. The observation records edit answers tied to the checked record, replacement authorization tokens, and returned values.*

d) Proof roadmap: Registration reflection follows by enumerating the finite protected-call traces and checking their order- and identity-preserving correspondence. The edit-rule checks and greatest-fixed-point argument prove exact checking. Every safe execution set is contained in the computed family, and a nonempty computed family yields a safe runtime implementation. Four explicit pairs of execution records prove the information result. Induction over the runtime rules proves repeated use. A case analysis over edits, protected calls, races with the rule update, returns, and recovery establishes the weak bisimulation. The appendix gives the complete witnesses and case proofs.

VII. Mechanization and Executable Validation

Six Lean modules pass `--trust=0` and prove the claims used by the paper. `exact_checker_iff_realization` proves that the executable checker accepts exactly when a safe execution set exists. `registered_workflow_exact_admission` combines typed registration, trusted checking of every protected call, and exact edit checking. `six_edit_derivation_exact` proves the executable derivation sound and complete for all six edit rules. `exact_edit_installs_trace_safe_monitor` constructs an atomic rule update whose every finite continuation preserves AgentSec. `fresh_alias_installation_serialized` proves both possible orders between a protected call and that update. `compilation_preload_preserves_agentSec` models preparation as an explicit transition and proves that preparing an inactive candidate preserves the same invariant.

All 19 paper-specific tests pass, together with 62 authority and compiler regression tests for supporting components. Across accepted instances with 2–128 outcomes, checker time is 0.11–53.47 ms. Rejected instances take 0.11–5.51 ms, and enumerating 6–720 unordered completions takes 0.33–50.35 ms. Section A-M reports the complete measurement protocol and executable coverage.

VIII. Related Work

a) Recovery, workflow update, and synthesis: Output-commit recovery reconstructs a fixed computation. Generalized nonblocking and Live Synthesis solve supplied transition systems and completion conditions, while workflow inheritance and dynamic controller update connect supplied old and new specifications [21], [22], [16], [17], [18], [23], [24], [25]. Our controller derives the edited workflow, action reuse, still-required outcomes, completion conditions, and new runtime rules directly from the execution record. It computes every safe continuation and puts the corresponding rules into effect atomically with new authorization or authorization reuse.

b) Agent recovery mechanisms: ACRFence blocks replay after Restore, DART selects checkpoints, Atomix delays external actions until commit, and Rebound makes rollback atomic and auditable [7], [15], [13], [26]. Our theorem gives one exact criterion for all six derived Agent edits, a rejection proof, and a lower bound proving that all four parts of the execution record are necessary. It carries these results through the atomic rule update and arbitrary finite execution.

IX. Conclusion

Safe execution editing has an exact criterion. Past protected actions remain unchanged, and every still-required outcome retains a policy-safe completion. For every registered Fork, Restore, Merge, or extension, our checker computes every safe continuation or returns a finite proof that no safe runtime implementation exists. The Exact Edit-Safety theorem connects this exact decision to the four necessary parts of the execution record, the atomic rule update, repeated execution edits, and equivalence with an ideal atomic machine. It provides a formal foundation for Agent runtimes that support Fork, Restore, and Merge.

APPENDIX A
SUPPLEMENTARY PROOF APPENDIX

This appendix instantiates the proof objects used by theorem 2 over finite registered call models and atomic policy-domain rule versions. Independent domains retain the product interpretation stated in the paper.

A. The Declarative Definition and the Computed Result

Fix a well-formed safety-check instance $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$ for which the edit derivation yields $\delta = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. All sets in this subsection retain outcome labels, so two equal resolved words belonging to different outcomes remain different elements.

Definition 4 (Required source outcomes)**.** *Write* $\mathsf{kind}(u)$ *for the constructor of* u*. Call either Fork constructor a Fork and either Restore constructor a Restore. For the matching source row in table I, define*

u	$\mathsf{SrcReg}(H,u)$
Fork	$\{(L,\mathcal{C}),(R,\mathcal{C})\}$
Restore	$\{(\mathsf{cur},\mathcal{C}),(k,\mathcal{C}_k)\}$
$\mathsf{MergeSelect}(g,w,\sigma)$	$\{(w,[\![T_w]\!]),(\bar{w},[\![T_{\bar{w}}]\!])\}$
$\mathsf{MergeJoin}(g,\sigma)$	$\{(L,[\![T_L]\!]),(R,[\![T_R]\!])\}$

The full set of source outcomes is

$$\mathsf{RequiredBefore}(H,u) = (\{\mathsf{ctx}\}\times O_{\mathrm{unchanged}}) \uplus \biguplus_{\substack{(r,\mathcal{D})\\ \in \mathsf{SrcReg}(H,u)}} (\{r\}\times O_{\mathcal{D}}).$$

Write $\mathsf{Done}_H(r,o)$ *when the recorded progress for region* r *shows that every event in outcome* o*'s pomset has completed. Only* RestoreReplace *treats completed current outcomes as finished:*

$$\mathsf{DoneCur}_H = \{(\mathsf{cur},o) \mid o\in O_{\mathcal{C}} \wedge \mathsf{Done}_H(\mathsf{cur},o)\},$$

u	$\mathsf{Satisfied}_{H,u}$
$\mathsf{RestoreReplace}(b,k,\sigma)$	$\mathsf{DoneCur}_H$
otherwise	$\varnothing$

Completion observability makes this current-role set either all of $\{\mathsf{cur}\}\times O_{\mathcal{C}}$ *or empty. The only sets eligible for signed removal are*

$$\mathsf{CurOut} = \{\mathsf{cur}\}\times O_{\mathcal{C}}, \qquad \mathsf{Other}_w = \{\bar{w}\}\times O_{[\![T_{\bar{w}}]\!]},$$

u	$\mathsf{CanRemove}(H,u)$
$\mathsf{RestoreReplace}(b,k,\sigma)$	$\mathsf{CurOut}\setminus \mathsf{Satisfied}_{H,u}$
$\mathsf{MergeSelect}(g,w,\sigma)$	Other_w
otherwise	$\varnothing$

$\mathsf{reqauth}_H(a)$ *follows* a*'s source region to the unique signed authority record in* G*, or to the stored copy in* K *when* a *comes from a checkpoint. Context identity, carry, and clone preserve that record, while every source region introduced by an edit rule receives the authority that signed the rule and policy version. Thus* $\mathsf{reqauth}_H$ *is total and immutable on* $\mathsf{RequiredBefore}(H,u)$*, including Fork-created* L/R *copies. For every* $a\in\mathsf{CanRemove}(H,u)$*, the removal request must contain a signature by* $\mathsf{reqauth}_H(a)$ *over*

$$\mathsf{rmmsg}(H,u) = (\omega,\kappa,\zeta,\sigma,\mathsf{fp}(H),\ \mathsf{CanRemove}(H,u),\ \mathsf{origin}(\mathsf{CanRemove}(H,u))).$$

Without all required signatures, the edit has no derivation. With them, $\mathsf{AuthorizedRemoval}_{H,u} = \mathsf{CanRemove}(H,u)$*. Write* $\mathsf{Sat}_u = \mathsf{Satisfied}_{H,u}$ *and* $\mathsf{Rm}_u = \mathsf{AuthorizedRemoval}_{H,u}$*. The checker requires*

$$\mathsf{StillRequired}_u = \mathsf{RequiredBefore}(H,u)\setminus(\mathsf{Sat}_u\uplus\mathsf{Rm}_u),$$
$$\mathsf{RequiredBefore}(H,u) = \mathsf{StillRequired}_u\uplus\mathsf{Sat}_u\uplus\mathsf{Rm}_u.$$

Definition 5 (Required-outcome preservation)**.**

$$\mathsf{PreservesRequired}(H,u,\widehat{B}) \iff \forall a\in\mathsf{StillRequired}_u.\ \exists(t,y)\in\widehat{B}.\ \mathsf{Covers}_u(t,a).$$

Here $\mathsf{AuthorizedRemoval}_{H,u}$ *contains the typed unselected-arm outcomes for* MergeSelect *or the current-role outcomes of an authorized* RestoreReplace*.* $\mathsf{Satisfied}_{H,u}$ *contains the completed current role of* RestoreReplace*. Both sets are empty for every other row and for a registered extension. Thus* $\mathsf{StillRequired}_u = \mathsf{RequiredBefore}(H,u)\setminus(\mathsf{Satisfied}_{H,u}\uplus\mathsf{AuthorizedRemoval}_{H,u})$*, so* PreservesRequired *covers exactly the source outcomes that are neither completed nor authorized for removal.*

Let $\mathfrak{R}_\Theta$ be the finite set of safe execution sets from definition 2, ordered componentwise by

$$(L_1,\widehat{B}_1)\sqsubseteq(L_2,\widehat{B}_2) \iff L_1\subseteq L_2 \ \wedge\ \widehat{B}_1\subseteq\widehat{B}_2.$$

Lemma 8 (The computed result is exact)**.** *The declarative predicate* $\mathsf{MaxSafe}(\Theta;L,\widehat{B})$ *holds for some pair if and only if* $\widehat{B}^\dagger_{H,u}\neq\varnothing$*. When such a pair exists, it is unique and equals* $(W^\dagger_{H,u},\widehat{B}^\dagger_{H,u})$*. This equality is a theorem relating two independently stated objects, rather than the definition of the ideal machine.*

Proof. Let $(L,\widehat{B})\in\mathfrak{R}_\Theta$. The workflow, outcome, and policy requirements give $\widehat{B}\subseteq\widehat{B}_0$. For every $(o,y)\in\widehat{B}$, every prefix $z\preceq_{\mathrm{pref}} y$ belongs to $L=\mathsf{Pref}(\pi_2\widehat{B})$. Persistent outcome coverage therefore supplies, for every $o'\in\mathsf{Compat}(z)$, a pair $(o',y')\in\widehat{B}$ extending z. Thus $\widehat{B}\subseteq\widehat{\Phi}(\widehat{B})$. By monotonicity on the finite lattice $2^{\widehat{B}_0}$, every post-fixed set is contained in the greatest fixed point, so $\widehat{B}\subseteq\widehat{B}^\dagger$. Taking projected prefixes yields $L\subseteq W^\dagger$.

Conversely, suppose $\widehat{B}^\dagger\neq\varnothing$. The fixed-point equality keeps every compatible outcome completable and, at ϵ, combines with the checked preservation maps to establish PreservesRequired. Inclusion in $\widehat{B}_0$ ensures that complete executions follow the target workflow and policy. Prefix closure of $\mathcal{A}_u$ makes every generated prefix safe, and $W^\dagger=\mathsf{Pref}(\pi_2\widehat{B}^\dagger)$ gives a completion after every prefix. Hence $(W^\dagger,\widehat{B}^\dagger)\in\mathfrak{R}_\Theta$, and the preceding containment makes it greatest. If two greatest elements existed, each would contain the other componentwise, so they would be equal. If $\widehat{B}^\dagger=\varnothing$, the first half places

every hypothetical safe execution set inside the empty set, contradicting its required nonemptiness. □

Corollary 1 (Exact checking)**.** *The ideal edit transition is enabled by the declarative* $\mathsf{SafeBehavior}(\Theta)$ *exactly when the checker returns a nonempty fixed point. On that edge, the ideal language* L^* *equals the generated language of* $\mathsf{Can}(W^\dagger, B^\dagger)$.

Proof. The first statement is lemma 8. The second combines that lemma with lemma 6. □

Proposition 2 (Output-sensitive checking)**.** *Fix the unit-cost symbolic-checker model with constant-time finite-map lookup and policy-automaton transitions. Let*

$$D = |\Theta|_{\mathrm{chk}}, \qquad n = \max_o |X_{p_o}|,$$
$$\Lambda = \sum_o \sum_{w \in \mathsf{Lin}(p_o)} (|w|+1), \quad N_B = |\widehat{B}_0|,$$
$$V_B = \sum_{(o,y)\in \widehat{B}_0} (|y|+1).$$
$$\mathcal{G}_{\mathrm{cov}} = \left\{ ((o,y), z, o') \;\middle|\; \begin{array}{l} (o,y) \in \widehat{B}_0,\ z \preceq y, \\ o' \in \mathsf{Compat}(z) \end{array} \right\}.$$

Here D *is the checked input size,* Λ *is the total size of the emitted linearizations, and* $\mathcal{G}_{\mathrm{cov}}$ *contains the candidate, prefix, and required-outcome triples examined by the checker. The checker runs in* $O(D + n\Lambda + |\mathcal{G}_{\mathrm{cov}}| + V_B \log(2+V_B))$ *time and* $O(D + \Lambda + |\mathcal{G}_{\mathrm{cov}}|)$ *space, including the data it returns. Moreover,* $N_B \le \sum_o |\mathsf{Lin}(p_o)|$, $|\mathcal{G}_{\mathrm{cov}}| \le N_B(n+1)|O_{\mathcal{C}_u}|$, *and the number of nonempty strict-removal ranks is at most* N_B.

Proof. A backtracking topological-order enumerator emits all indexed linearizations in $O(n\Lambda)$ time, while resolution, policy simulation, and construction of raw and resolved prefix tries are linear in emitted volume. For every support key (z, o'), maintain the number of remaining pairs (o', y') extending z, initially enqueue candidates having a required outcome with zero support, and remove candidates in synchronous batches. When removal makes a support count zero, enqueue exactly the candidates that depend on that requirement for the next batch. Each candidate and each triple in $\mathcal{G}_{\mathrm{cov}}$ is processed at most once, and the batches are exactly $\widehat{B}_i \setminus \widehat{B}_{i+1}$. Store one rank and cause per removed candidate rather than copied sets, and construct $\mathsf{Can}(W, B)$ by bottom-up sorting of finite-trie derivative signatures in $O(V_B \log(2+V_B))$ time. □

B. Ordered Rejection Proofs

For an empty fixed point, the returned proof is

$$C_{\mathrm{fp}} = (\widehat{B}_0, E_0, \ldots, E_k), \qquad E_i \subseteq \widehat{B}_i \times \mathsf{Pref}(\pi_2 \widehat{B}_i) \times O_{H,u}.$$

The verifier first recomputes the canonical $\widehat{B}_0(\Theta)$ and requires equality with the serialized first component. An entry $((o,y), z, o') \in E_i$ verifies exactly when $z \preceq y$, $o' \in \mathsf{Compat}(z)$, and no $(o', y') \in \widehat{B}_i$ extends z. The verifier requires the set of first components in E_i to equal $\widehat{B}_i \setminus \widehat{\Phi}(\widehat{B}_i)$, sets $\widehat{B}_{i+1} = \widehat{B}_i \setminus \pi_1(E_i)$, and accepts only when $\widehat{B}_{k+1} = \varnothing$. All sets, prefixes, and compatibility checks are finite and use the canonical order fixed by the checker.

Lemma 9 (The rejection proof is sound and complete)**.** *The verifier accepts* C_{fp} *iff the descending fixed-point computation reaches* $\widehat{B}^\dagger = \varnothing$. *In that case no safe execution set exists.*

Proof. The initial equality check ties the proof to Θ, and every verified round is exactly $\widehat{B}_{i+1} = \widehat{\Phi}(\widehat{B}_i)$, so canonical iteration supplies completeness. For soundness, induction places every post-fixed family inside every $\widehat{B}_i$, so an empty final family excludes every nonempty safe execution set. □

C. Exact Returned Proof Data

Let enc be canonical and length-delimited, with κ authenticating $\mathcal{A}$. The symbolic LTS uses domain-separated injective authenticated digests $\mathsf{Digest}_{\mathrm{rules}}$ and $\mathsf{Digest}_{\mathrm{record}}$, so equal digests have equal encoded inputs. For $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$, define

$$\gamma(\Theta) = \mathsf{Digest}_{\mathrm{record}}(\mathsf{enc}(\mathsf{request}, \omega, \zeta, \kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)),$$
$$H_{\mathsf{aut}} = \mathsf{Digest}_{\mathrm{rules}}(\mathsf{enc}(\mathsf{aut}_Z, \widehat{B}^\dagger_{H,u})).$$

The digest checked before the rules change is

$$\mathsf{cut}_Z = \mathsf{Digest}_{\mathrm{record}}(\mathsf{enc}(\omega, \zeta, \kappa, H, \Delta, \mathsf{out}, u, \mathcal{C}_{H,u}, H_{\mathsf{aut}}, \eta^-, \eta^+)). \tag{11}$$

Because H contains $\chi, \mathsf{ver}, \rho, \beta$, the digest covers workflow progress, record version, registry contents, active rule entries and versions, the authorization log, and the request queue. The verifier re-derives the edit-rule judgment and complete fixed point labeled by outcome, reconstructs $(W^\dagger, B^\dagger)$, verifies both digests, and checks the full automaton isomorphism

$$\mathsf{aut}_Z \cong \mathsf{Can}(W^\dagger, B^\dagger),$$

including its initial state, marking, and every labeled transition. Canonical encoding order makes Invalid, Reject, and Installable unique and unchanged by consistent renaming of internal identifiers. A byte implementation may authenticate enc with signatures or collision-resistant digests. The byte-level guarantee is parameterized by signature unforgeability and digest collision resistance, represented as authenticated identities in the formal model.

D. Proofs for Edit Rules and Rechecking

1) Edit-rule correctness:

Full proof of lemma 2. Unique object and edit-rule identifiers select one row and record, while context identity preserves everything outside the edited region. The Fork rows check both copies. RestoreReplace checks the checkpoint copy and the exact treatment of the current continuation, while RestoreLive checks both the retained current continuation and the checkpoint copy. MergeSelect checks the selected and unselected regions, while MergeJoin checks both retained regions before its barrier. Clone and carry supply the global bijections and preserve which workflow calls belong to each outcome and which authority controls each source region. Constructor induction defines pr^u_r,

proves the cover, and supplies every Covers_u witness. The common derived record fixes all remaining fields, preserves or initializes branch progress, and assigns each source region introduced by an edit rule its signed authority record. Fresh identifiers, partial-constructor checks, and lemma 1 preserve uniqueness, acyclicity, and contract well-formedness. Finally, $\beta'(x) = (j_x^+, \eta^+)$ and $(x, j_x^+, h_x^+) \in \mathcal{H}_u^+$ for every current target workflow call x. □

2) Registered extension: The Agent may name only an immutable record ξ that specifies the exact prior edit rules, registry, execution record, and policy, together with finite disjoint additions $D_\Sigma, D_\rho, D_\alpha$ and a successor policy $\mathcal{A}^+$. The set D_α contains the signed authority record for every newly introduced outcome and its source region. The policy authority and call authorization service verify ξ. Existing meanings remain unchanged, new identifiers are fresh and domain-scoped, and $\mathcal{A} \subseteq \mathcal{A}^+$. For fresh η^+, define

$$\begin{aligned}
G^+ &= G \uplus D_\alpha,\\
\Sigma_{\zeta^+} &= \Sigma_\zeta \uplus D_\Sigma,\\
\rho^+ &= \rho \uplus D_\rho,\\
H_\xi^+ &= (\omega, G^+, T, K, \Sigma_{\zeta^+}, \rho^+, \beta^+,\\
&\qquad \chi, \mathsf{ver}+1, \eta^+),\\
\mathcal{C}_\xi &= [\![T]\!],\\
\mathsf{RequiredBefore}(H, \xi) &= \mathsf{StillRequired}_\xi,\\
\mathsf{StillRequired}_\xi &= O_{[\![T]\!]},\\
\mathsf{Satisfied}_{H,\xi} &= \varnothing,\\
\mathsf{AuthorizedRemoval}_{H,\xi} &= \varnothing,\\
\mathsf{Covers}_\xi(t, o) &\iff t = o.
\end{aligned}$$

The checked judgment is

$$(\kappa, H, \mathcal{A}) \vdash \mathsf{Extend}(\xi) \Downarrow (\kappa^+, H_\xi^+, \mathcal{A}^+, \mathcal{C}_\xi, \eta, \eta^+).$$

It preserves $T, K, \chi, \Delta, \mathsf{out}$ and every existing row of G, Σ_ζ, ρ, appends D_α to the execution record, and authorizes no removal. Every current outcome, workflow call, action, source region, authority record, and causal edge therefore remains unchanged. It uses the same checker and atomic rule update as an execution edit.

Lemma 10 (Registered extension preserves safety)**.** *For a verified* $\mathsf{Extend}(\xi)$*, the current safe executions after recorded progress form a post-fixed family for the extension instance and are therefore contained in* $\widehat{B}^\dagger_{H,\mathsf{Extend}(\xi)} \neq \varnothing$*. Consequently, a valid registered extension cannot remove a currently required outcome or leave it without a safe completion.*

3) Rechecking after a prefix:

Full proof of lemma 5. Because z prefixes a projected member of $\widehat{B}^\dagger$, raw recoverability and streaming in lemma 3 give the recoverability and authorization-log equalities. Removing the executed prefix from the outcome-labeled executions retains every compatible outcome and gives

$$\mathsf{Compat}_{\mathcal{C}/\mathsf{raw}(z)}(s) = \mathsf{Compat}_{\mathcal{C}}(zs).$$

Together with authorization-sequence concatenation, streaming resolution yields

$$\widehat{B}_0(\mathcal{C}, \Delta)/z = \widehat{B}_0(\mathcal{C}/\mathsf{raw}(z), \Delta_z),$$

with the same retained outcome identities, policy, and target registry. Thus $\widehat{B}^\dagger/z$ is post-fixed for the residual operator and lies within its greatest fixed point Y. Conversely, streaming and residual policy safety put $zY = \{(o, zy) \mid (o, y) \in Y\}$ inside the original $\widehat{B}_0$. The union $\widehat{B}^\dagger \cup zY$ is post-fixed because prefixes $t \preceq z$ use existing witnesses, while prefixes $t = zs$ use witnesses in Y. Greatestness gives $zY \subseteq \widehat{B}^\dagger$, hence $Y = \widehat{B}^\dagger/z$. Finally, $\mathsf{Pref}(B)/z = \mathsf{Pref}(B/z)$ yields the generated-language equality. □

Proof of lemma 10. The verified extension preserves the current workflow, which calls refer to the same action, every outcome label, and $\mathcal{A} \subseteq \mathcal{A}^+$. Every earlier safe completion and coverage witness therefore remains valid for the next fixed-point computation. The current residual family is therefore post-fixed, and greatestness gives its inclusion in a nonempty successor fixed point. □

E. Lean Theorem Map

Six Lean modules machine-check the claims used in the paper. `FiniteCore` proves the executable greatest-fixed-point checker equivalent to the declarative safe-execution-set predicate, including streaming resolution and greatestness. `RegistrationRefinement` proves that registration accepts the typed finite model, every registered call remains under trusted control, action identity is preserved, and the finite and semantic contracts agree. `HistoryStructure` proves `deriveEdit_iff`, well-formedness preservation, and executable fixtures for Choice/Parallel Fork, Replace/Live Restore, Select/Join Merge. `OperationalSemantics` proves invariant preservation across every runtime trace, closure of all six edits, registered extension safety, and both use–installation race orders. `CompilationBridge` constructs a secure atomic installation from executable admission and reflects every successful installation back to admission. `PaperTheorems` composes these results into exact registered admission, six-edit derivation, compilation-preload preservation, trace-safe installation, and use–installation serialization theorems. The general pomset lift, information lower bounds, and ideal–runtime weak bisimulation complete the theorem in this appendix.

F. Normalized Evidence and Safe Projection

This subsection asks which parts of the authenticated security state are necessary to reproduce every safety-check and installation answer. We first normalize private identifiers so that representation choices cannot distinguish two states, then remove one information factor at a time and construct queries whose correct answers differ.

We make the normalization implicit in Sec explicit. Let $\mathcal{Q}_{\mathrm{sec}}$ pair every valid $V = \mathsf{Sec}(S; c, r)$ with a well-typed $q ::= \mathsf{Decide}(u) \mid \mathsf{TryInstall}(u, Y)$, permitting stale

names. For a full view $V = \mathsf{Sec}(S; c, r)$, set $\Theta_V(u) = \Theta_S(u)$ and $\mathsf{Ready}_V(u, Y) = \mathsf{Ready}(S, u, Y)$. Ans returns $\mathsf{Compile}(\Theta_V(u))$ for Decide, and for TryInstall returns the successor bundle exactly when $\mathsf{Ready}_V(u, Y)$, otherwise NoCommit. An equivariant information map f is exact iff some D_f maps every $\langle f(V), q\rangle_{\cong}$ to $[\mathsf{Ans}(V, q)]_{\cong}$. Let $\mathcal{K}$ be the finite typed key universe at a checked source snapshot, containing tagged outcome, call-position, action-ID, authorization-record, checkpoint, group, domain, rule-version, policy-version, and schema-version keys. Public keys comprise policy-authority identifiers and other identifiers declared external by the interface; every other key may be consistently renamed. For a typed query q, $\mathsf{supp}(q)$ is defined recursively over its full syntax, including all identifiers nested inside a supplied symbolic certificate, automaton, or source-snapshot seal. Let $\mathcal{F}$ be the finite set of typed field paths and assign every semantic base field a unique owner

$$\mathsf{own} : \mathcal{F}_{\mathrm{base}} \longrightarrow \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

$\mathbf{P}$ owns workflow constructor tags, current/checkpoint/target contract nodes, order and outcome membership, certified-origin authority rows, schema kind and source fingerprints, lift relations, and authorized-removal declarations. $\mathbf{I}$ owns the call–action map, action-ID–canonical-request/label/scope rows, call-position registered sources, and logical rule-entry and authorization-token slots. $\mathbf{E}$ owns the global resolved trace, current and checkpoint cursors, authorization-record links, retry paths, stored return records, dispatch-queue status, and remaining policy. $\mathbf{C}$ owns the domain, policy/schema/view versions, private namespace, rule-version allocation and phase, current rule-entry ownership, certificate origin and coordinates, and checkpoint snapshot coordinates. These four lists are disjoint and exhaustive for semantic base fields. Certificate bytes, seals, fingerprints, digests, derived authorization sequences, and foreign-key paths are derived fields rather than additional semantic facts.

Write $f \rightsquigarrow g$ when the value or well-typed interpretation of derived field g depends immediately on field f. The dependency graph is acyclic because canonical encodings are constructed from base relations in a fixed order. Every foreign-key path depends on its target. Authenticated rule-entry and authorization-token bytes depend on their $\mathbf{I}$ slot and $\mathbf{C}$ rule version and owner. Authorization record reconstruction and the authorization sequence depend on the $\mathbf{E}$ authorization record and cursor rows together with their $\mathbf{I}$ action IDs and labels. Target certificates, source-snapshot seals, and digests depend on every encoded base field in their payload. A normalized full view V is a compatible finite assignment to all base fields together with the unique values of all derivable fields. Compatibility requires equal values on shared typed keys and the well-formedness conditions of definition 1. The natural join $\mathbf{P} \bowtie \mathbf{I} \bowtie \mathbf{E} \bowtie \mathbf{C}$ is the compatible union of these uniquely owned base fields followed by deterministic recomputation of derived fields. It is undefined precisely when shared typed keys disagree or a required foreign-key target is absent.

For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, start with base fields owned by J and the public sort declarations needed to type the retained records. Repeatedly delete any foreign-key path whose target has been deleted and any derived field having a deleted transitive dependency. The unique fixed point of this deletion is $\mathsf{Ret}(J)$, and $\pi_J(V)$ is V restricted to $\mathsf{Ret}(J)$. This definition prevents an erased fact from surviving inside a seal, digest, archived authorization record-only registry path, or certificate payload.

The incidence-erasure projection $\pi_{\neg oc}$ starts from the full view, deletes the call–action map, and applies the same dependency deletion while retaining the call-position and action-ID marginals. The projection $\pi_{\neg ar}$ analogously deletes the authorization-record–action map and every path that reconstructs it, including authorization-record–creator, cursor–authorization-record, archived creator–action-ID paths, and every degree, count, or adjacency summary derived from those paths, while retaining the authorization-record, creator, call-position, and action-ID marginals. State views are compared modulo a sort-preserving bijection on private keys that fixes all public keys. For possibly different literal queries q, q', write $(V, q) \cong (V', q')$ when one such bijection maps the full syntax of q to that of q'; equivalently, $\langle V, q\rangle_{\cong} = \langle V', q'\rangle_{\cong}$.

Lemma 11 (Projection is well defined)**.** *Natural join, dependency deletion, and private renaming commute. Consequently, $[\pi_J(V)]_{\cong}$, $[\pi_{\neg oc}(V)]_{\cong}$, and $[\pi_{\neg ar}(V)]_{\cong}$ do not depend on a representative or on a certificate serialization.*

Proof. A sort-preserving bijection preserves key equality, foreign-key reachability, ownership, and the dependency relation. It therefore maps each deletion round to the corresponding deletion round in the renamed view. Induction on the finite acyclic dependency graph proves that the retained fixed points correspond. Canonical recomposition is equivariant because it is a typed constructor over exactly those retained fields. □

Lemma 12 (Representation-independent observational separation)**.** *Let V_0, V_1 be finite valid full views and let q_0, q_1 be typed queries. If $[\mathsf{Ans}(V_0, q_0)]_{\cong} \neq [\mathsf{Ans}(V_1, q_1)]_{\cong}$, then every exact information map f distinguishes the joint inputs: $(f(V_0), q_0) \ncong (f(V_1), q_1)$. The claim is representation-independent over arbitrary encodings and characterizes the semantic information factors.*

Proof. Assume toward contradiction that $(f(V_0), q_0) \cong (f(V_1), q_1)$. The joint decoder inputs $\langle f(V_0), q_0\rangle_{\cong}$ and $\langle f(V_1), q_1\rangle_{\cong}$ then coincide. Exactness forces equal answer orbits, contradicting the premise. □

G. Bootstrap States and Explicit Separation Pairs

Definition 6 (Checked registration refinement)**.** *A boundary-finite typed Agent workflow $\mathcal{W}$ is a finite-state outcome-labeled LTS whose complete paths end at outcome terminals and whose τ-erased protected-call projections $\partial e = (o, s_1 \cdots s_n)$ form finitely many classes under equality $\equiv_\partial$.* CheckReg *rejects any reachable strongly connected component that can still reach*

Erased	Full-view pair	Exact answers
$u_R(\mathcal{C}) := \mathsf{RestoreReplace}(b, k, \sigma)$ with $\mathcal{C}_k = \mathcal{C}$.		
$\mathbf{P}$	$q^i_{\mathbf{P}} = \mathsf{MergeSelect}(g_i, L, \sigma)$; $T^0 = b_x{:}X \oplus^{\mathrm{open}}_{g_0} b_y{:}Y$, $T^1 = b_x{:}X \parallel_{g_1} b_y{:}Y$; joint inputs are isomorphic after erasure.	V^0: realize; V^1: Invalid (wrong mode).
$\mathbf{I}$	$q_{\mathbf{I}} = \mathsf{RestoreLive}(b_L, k, \sigma_{\mathbf{I}})$ after safe empty-root/ForkChoice/Save; $\Delta = \epsilon, \mathcal{A} = \mathsf{Pref}\{a\}$; $q^0_{\mathrm{act}}(x) = q^0_{\mathrm{act}}(y) = d_0$; $q^1_{\mathrm{act}}(x) = d_0 \neq d_1 = q^1_{\mathrm{act}}(y)$; $\ell(d_0) = \ell(d_1) = a$.	$(X \otimes Y_k) \oplus Y$: one new, accept; two new reject aa.
$\mathbf{E}$	$q_{\mathbf{E}} = \mathsf{TryInstall}(u_R(x), Y_0)$; Y_0 is compiled at V^0 after safe empty-root/ForkChoice/Save/new; $V^0 \xrightarrow{\mathsf{Dispatch}(d)} V^1$, changing only the release phase.	V^0: install; V^1: NoCommit (source snapshot changed).
$\pi_{\neg ar}$	$q_{\neg ar} = u_R(x)$ after safe empty-root/ForkChoice/Save/new; $q_{\mathrm{act}}(x) = d$, $\mathcal{A} = \mathsf{Pref}\{a\}$; same marginals $\{c, x\}, \{p\}, \{d, e\}$; $(q_{\mathrm{act}}(c), p) = (d, d)$ in V^0, (e, e) in V^1.	$\mathsf{reuse}(x_k, d)$: accept; $\mathsf{new}(x_k, d)$: reject aa.
$\mathbf{C}$	$q_{\mathbf{C}} = \mathsf{TryInstall}(u_0, Y_0)$, with full package Y_0 compiled at V^0, is literal in both; name supplies agree off the rule-version sort, while current rule version, ownership, and dependent seals differ.	V^0: install; V^1: NoCommit.

TABLE II
FACTOR- AND INCIDENCE-ERASURE PAIRS WITH ISOMORPHIC RETAINED VIEWS AND OPPOSITE EXACT ANSWERS.

an outcome and contains a protected-call edge, permits erased τ-cycles, and then enumerates the finite classes. $\mathcal{R}$ stores a candidate map λ, and $\mathsf{BRuns}(\mathcal{R})$ is the finite outcome-indexed language obtained by linearizing its registered root pomsets and labeling every workflow call by its protected Use. *$\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$ means that λ induces a bijection from boundary classes to $\mathsf{BRuns}(\mathcal{R})$ that preserves outcomes, incidence, order, and the signed policy authority for each outcome's registered source. The check also requires protected actions to be exactly the equivalence classes of canonical requests, requires λ to be total, unique, and free of raw instructions, and requires authorization record evolution to commute with* Resolve *without dropping a required outcome, workflow call, or authorization record.*

Every separation proof follows the same recipe. Starting from checked initial states, we vary exactly one security-state factor, keep the projected joint state–query inputs isomorphic, and obtain opposite correct answers; every literal query is fixed except the $\mathbf{P}$ pair's private group renaming. The resulting pair proves that an exact abstraction cannot erase that factor.

We construct the finite lower-bound witnesses from one literal empty initial state $\mathsf{S}_\emptyset$. A registered call model $\mathcal{R}$ compiled from a boundary-finite typed Agent workflow $\mathcal{W}$ contains only finite contracts, schemas, policy, the checked registration map, and authenticated outcome-authority, workflow-call, protected-action, canonical-request, certified-origin, scope, and label rows. It contains no authorization record, cursor, dispatch-queue entry, rule version, certificate, return record, automaton state, or installation record. The registration relation has a step $\mathsf{S}_\emptyset \xrightarrow{\mathsf{register}(\mathcal{W},\mathcal{R},n)} \mathsf{Reg}(\mathcal{R}, n)$ exactly when $\mathsf{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts and n is a fresh private namespace, followed by checked root installation. Root installation constructs empty authorization record, cursor, and dispatch-queue state, sets $\mathsf{ver} = 0$, activates one rule version, installs the canonical automaton, and authenticates every current binding and root outcome-authority row.

Lemma 13 (Registration reflection)**.** *For finite-state $\mathcal{W}$ and finite $\mathcal{R}$,* $\mathsf{CheckReg}(\mathcal{W}, \mathcal{R})$ *accepts iff the registered λ witnesses $\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$ in definition 6. On acceptance, the λ-lowerings of source executions modulo $\equiv_\partial$ correspond bijectively to* $\mathsf{BRuns}(\mathcal{R})$. *The correspondence preserves indexed outcomes, causal order, canonical-request equivalence, authorization record evolution, and the safety-check answer.*

Proof. The SCC check terminates because the state graph is finite. A reachable SCC that can still reach an outcome and contains a protected-call edge can be repeated before that outcome, producing projected words of unbounded length. Conversely, if no such SCC contains a protected-call edge, contracting the SCCs yields a DAG with a finite, enumerable projected language. The checker compares that language with $\mathsf{BRuns}(\mathcal{R})$ and directly checks outcome/workflow-call incidence, order, protected-action attestation, total unique registration, exclusion of unregistered calls, and authorization record evolution at every prefix. Therefore it accepts exactly the semantic conjunction defining $\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$, rather than an oracle restatement. Boundary projection erases only internal steps, so matching projected paths induce a bijection between the corresponding executions. Canonical-request equality and deterministic Resolve make authorization record evolution commute with this bijection at every prefix. The safe implementation families are therefore order-isomorphic, so the fixed-point operator preserves the safety decision, greatestness, and pullback of the automaton. □

Lemma 14 (Natural bootstrap)**.** *If* $\mathsf{CheckReg}(\mathcal{W}, \mathcal{R})$ *accepts and $\mathcal{R}$'s registered root contract and policy have a nonempty greatest safe implementation, registration and checked root installation reach a finite state $S_\mathcal{R}$ with* $\mathsf{AgentSec}(S_\mathcal{R}; c_\mathcal{R}, \epsilon)$. *The installation record originates at the checked root derivation, while every later replacement records an execution-edit derivation or a registered extension.*

Proof. Registration contributes only the checked $\mathcal{R}$ and n, and deterministic allocation constructs every runtime identifier from them. The root judgment and checked lifts establish core/schema coherence, and the recomputed nonempty fixed point establishes required-outcome coherence. Empty runtime progress establishes execution coherence, the canonical automaton at q_ϵ establishes automaton coherence, and the atomic phase/binding assignments establish rule-version coherence. Only initial installation creates a root origin, and every successful later installation stores its checked execution-edit or registered extension derivation. □

The following finite pairs use private names except for each displayed query and authorization labels. Each displayed state is reached from $\mathsf{S}_\emptyset$ by the stated registration, root-installation,

Save, protected-call, and edit steps; omitted fields are identical up to the stated private renaming and dependency deletion.

a) $\mathbf{P}$*: workflow structure:* Let X and Y be singleton contracts with distinct workflow calls x and y, a shared protected action, and a universal residual policy. From the same registered singleton root, checked initial installation followed by ForkChoice reaches $V^0_{\mathbf{P}}$ with private group g_0, while the corresponding ForkParallel trace reaches $V^1_{\mathbf{P}}$ with private group g_1. The two views differ only in

$$V^0_{\mathbf{P}}.T = b_L{:}X \oplus^{\mathsf{open}}_{g_0} b_R{:}Y, \qquad V^1_{\mathbf{P}}.T = b_L{:}X \parallel_{g_1} b_R{:}Y.$$

Use the same registered MergeSelect schema with $q^i_{\mathbf{P}} = \mathsf{MergeSelect}(g_i, L, \sigma)$. The choice view has a unique edit derivation and a nonempty one-step greatest safe implementation, whereas the parallel view has no MergeSelect derivation because its group mode is wrong. Deleting $\mathbf{P}$ removes the constructor tag and every authenticator depending on it; the private renaming $g_0 \mapsto g_1$ then makes the retained state–query pairs isomorphic.

b) Bootstrap convention for the incidence pairs: Let $\mathbf{1}$ be the singleton-outcome contract whose unique pomset is empty. Its completion ϵ gives a nonempty greatest safe implementation under $\mathcal{A} = \mathsf{Pref}\{a\}$. Both pairs below install this safe root and then a checked ForkChoice. Each inactive schema row needed to equalize protected-action marginals is identical in both views and never becomes active.

c) $\mathbf{I}$*: workflow call–protected-action incidence:* Let X, Y be singleton contracts with workflow calls x, y, and let d_0, d_1 be protected actions without authorization records, both labeled a. Two checked registered call models have identical topology, schemas, workflow call marginals, and protected-action marginals but the following incidence:

	$q_{\mathrm{act}}(x)$	$q_{\mathrm{act}}(y)$
$V^0_{\mathbf{I}}$	d_0	d_0
$V^1_{\mathbf{I}}$	d_0	d_1.

From the installed $\mathbf{1}$ root, the common Fork schema derives $b_L{:}X \oplus^{\mathsf{open}}_g b_R{:}Y$. Each outcome uses one protected action, so both Forks install under $\mathsf{Pref}\{a\}$. Save b_R before progress and issue the common query $q_{\mathbf{I}} = \mathsf{RestoreLive}(b_L, k, \sigma_{\mathbf{I}})$. Up to clone renaming, the target is $(X \otimes Y_k) \oplus Y$. In $V^0_{\mathbf{I}}$, the left outcome uses one new and one reuse on d_0, while the bypass outcome uses one new, so every outcome has authorization sequence a. In $V^1_{\mathbf{I}}$, every left-outcome linearization uses new on both d_0, d_1, so its word is $aa \notin \mathcal{A}$. The left outcome is compatible with ϵ, so the first $\widehat{\Phi}$ round also removes the otherwise safe bypass completion and leaves the greatest fixed point empty. Erasing workflow call–protected-action incidence and its dependent rows makes the retained views isomorphic while preserving equal marginals.

d) $\mathbf{E}$*: authorization and execution progress:* Let C, X be singleton contracts with workflow calls c, x, let $q_{\mathrm{act}}(c) = q_{\mathrm{act}}(x) = d$, and label d by a. From the safe $\mathbf{1}$ root, a checked Fork installs $b_L{:}C \oplus^{\mathsf{open}}_g b_R{:}X$; save b_R, then use c to complete the left arm, create authorization record p, and prepare its dispatch-queue item. At the resulting $V^0_{\mathbf{E}}$, compile $u_{\mathbf{E}} = \mathsf{RestoreReplace}(b_L, k, \sigma_{\mathbf{E}})$ to the full package Y_0; its clone of x aliases d, so Y_0 is installable under $\mathcal{A} = \mathsf{Pref}\{a\}$. Let $V^0_{\mathbf{E}} \xrightarrow{\mathsf{Dispatch}(d)} V^1_{\mathbf{E}}$, which changes only $\mathbf{E}$'s dispatch-queue release phase and preserves $\mathbf{P}, \mathbf{I}, \mathbf{C}$ literally. For the common literal $q_{\mathbf{E}} = \mathsf{TryInstall}(u_{\mathbf{E}}, Y_0)$, $V^0_{\mathbf{E}}$ installs, whereas $V^1_{\mathbf{E}}$ returns NoCommit because the source-snapshot seal binds the queue phase.

e) $\pi_{\neg ar}$*: authorization record–protected-action incidence:* For a separate pair, keep the common active row $q_{\mathrm{act}}(x) = d$, creator c, authorization record p, and protected actions d, e labeled a, but set $q_{\mathrm{act}}(c) = d$ and $p \mapsto d$ in $V^0_{\neg ar}$, versus $q_{\mathrm{act}}(c) = e$ and $p \mapsto e$ in $V^1_{\neg ar}$. Both states follow the same safe empty-root/ForkChoice/Save/new shape above, have authorization sequence a, and use the common query $q_{\neg ar} = \mathsf{RestoreReplace}(b_L, k, \sigma_{\neg ar})$. The cloned x_k is $\mathsf{reuse}(x_k, d)$ in $V^0_{\neg ar}$, but $\mathsf{new}(x_k, d)$ produces forbidden word aa in $V^1_{\neg ar}$. The two views have the same creator, authorization record, workflow call, and protected-action marginals and the same retained incidence $x \mapsto d$. $\pi_{\neg ar}$ deletes the differing archived row for c, the authorization record incidence, and every derived reconstruction path, so the retained views are isomorphic. This proves the necessity of semantic authorization record–protected-action incidence, not of any particular authorization record-table column.

f) $\mathbf{C}$*: installation coordinates:* Let a bootstrap namespace be a family $n = (n_s)_{s \in \mathsf{Sort}}$ of injective, per-sort fresh supplies. From the same literal empty initial state, register the same public call model $\mathcal{R}$ with supplies n^0, n^1 that agree on every non-rule-version sort but mint distinct initial versions $\eta_0 \neq \eta_1$, and perform checked root installation. The resulting $V^0_{\mathbf{C}}, V^1_{\mathbf{C}}$ have literally identical $\mathbf{P}, \mathbf{I}, \mathbf{E}$ base fields, while $\mathbf{C}$'s active rule version, rule-entry ownership, certificate coordinates, and dependent authenticators differ. Compile the same registered edit u_0 at $V^0_{\mathbf{C}}$ to obtain $Y_0 = \mathsf{Installable}(\delta_0, W_0, \widehat{B}_0, Z_0)$, and submit the same literal query $q_{\mathbf{C}} = \mathsf{TryInstall}(u_0, Y_0)$ to both views. At $V^0_{\mathbf{C}}$, recomputation reproduces cut_{Z_0}, so installation succeeds. At $V^1_{\mathbf{C}}$, recomputation binds η_1 and its owner rather than the η_0-coordinates sealed in Z_0, so installation returns NoCommit. Erasing $\mathbf{C}$ recursively erases rule-version-dependent entry/token rows, certificate coordinates, seals, and authenticators; the identity bijection on every retained sort then makes the retained joint state–query inputs isomorphic.

Proposition 3 (Explicit factor and incidence lower bounds)**.** *Every pair above consists of finite states reached from the common empty initial state and satisfying* AgentSec*. Each pair has isomorphic projected joint state–query inputs and opposite answer orbits. Therefore every proper factor projection,* $\pi_{\neg oc}$*, and* $\pi_{\neg ar}$ *is inexact. Every exact abstraction must distinguish all displayed pairs.*

Proof. Lemma 14 gives the common initial state and valid root installation, while the explicit traces above use only modeled Save, protected-call, and execution-edit transitions. For the incidence pairs, the empty root completes with ϵ, and each installed Fork arm has a one-new completion labeled a. Hence

every displayed pre-query state is reachable under $\mathsf{Pref}\{a\}$. Open choice enables the Save steps, and the active automata enable each stated left-arm new before that arm becomes a completed, still-addressable continuation. For the $\mathbf{E}$ pair, Dispatch preserves $\mathbf{P}, \mathbf{I}, \mathbf{C}$ but changes a queue phase bound by Y_0's source-snapshot seal, giving install versus NoCommit. The displayed schema and resolution calculations give the other opposite answers. Lemma 11 and the specified private-name bijections give joint-input projection equality; for the $\mathbf{P}$ pair the bijection maps the support of $q^0_{\mathbf{P}}$ to that of $q^1_{\mathbf{P}}$, and for every other pair it fixes the common query support. Any projection omitting at least one factor collapses the corresponding factor pair, while the two incidence erasures collapse their corresponding incidence pairs. Inexactness and the general information lower bound now follow from lemma 12. □

Assigning the same caller-proposed target transition system, completion conditions, and rule version to the $\mathbf{P}$ pair makes their target-only data equal while leaving the authenticated queries and opposite answers unchanged. Thus target-only pre-derivation state is not exact, although generalized nonblocking is exact as a backend after trusted derivation supplies the plant, outcome identities, and conditional markings.

H. Ideal Atomic Execution Machine

An ideal state $I = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, c, r)$, with $c = (H_c, u, L^*, \widehat{B}^*)$, is exactly the authenticated execution data, current rule version and entries, installed specification, and resolved prefix. Write $\Theta_I(u) = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$. Compilation, automaton representation, transaction phases, authenticators, preloading, and installation microsteps refine this state through the compiled abstraction.

a) Edit transition: For current $\Theta_I(u)$, an undefined derivation gives state-preserving Invalid, while a defined derivation with no MaxSafe witness gives state-preserving Reject. Otherwise $\mathsf{Derive}(\Theta_I(u))$ supplies the target and rule version, and independently defined $\mathsf{SafeBehavior}(\Theta_I(u)) = (L^*, \widehat{B}^*)$ enables one atomic $\mathsf{Edge}(u, [\mathcal{H}^+_u]_{\cong})$ that installs them and resets r to ϵ. All three answers bind the same current source-snapshot token and are mutually exclusive and exhaustive.

b) Protected call: For submitted tool request m, an enabled x passes the ideal automaton exactly when its current entry and authorization token bind protected action d, $\mathsf{canon}_\kappa(m) = \mathsf{inv}_\rho(d)$, and $ra \in L^*$, where $a = \mathsf{new}(x, d)$ if d has no authorization record and $a = \mathsf{reuse}(x, d)$ otherwise. One atomic transition applies HStep, appends a to χ, advances ver, removes x's binding, and sets $r := ra$; new also creates the labeled authorization record and queues the canonical request, whereas reuse links to the existing return record. Any failed premise returns $\mathsf{deny}(x)$ without changing state or inventing a return value.

c) Retry, release, and crash: Save, authenticated retry, Dispatch, settlement, and recovery after a crash have the same state changes as their compiled rules in table III; compiler computation is absent. This least-generated LTS represents post-Dispatch external-network completion as environmental behavior.

By lemma 4, every reachable partial execution of the ideal machine retains a policy-safe completion until the next edit transition changes the workflow contract. The compiled runtime implements this machine without consulting the full completion set at every call.

I. Compiled State and Atomic Transitions

For $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$, the checker returns exactly one of

$$\mathsf{CheckOutput}(\Theta) ::= \mathsf{Invalid}([\gamma, C_{\mathrm{str}}]_{\cong}) \mid \mathsf{Reject}([\gamma, C_{\mathrm{fp}}]_{\cong}) \mid \mathsf{Installable}(\delta, W^\dagger, \widehat{B}^\dagger, Z),$$

where

$$Z = (\omega, \zeta, \kappa_Z, \mathsf{cut}_Z, u, H_{\mathsf{prog}}, \eta^-, \eta^+).$$

Invalid carries a verified undefined-derivation transcript, Reject carries the complete ranked elimination certificate, and Installable carries the unique derivation, coverage witnesses, canonical automaton, and Z. Z authenticates the complete source snapshot, derived target, automaton, and both old and new rule versions.

The compiled state for one policy domain is

$$S = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, \mathsf{phase}, \mathsf{bind}, \mathsf{prog}, \mathsf{q}, \mathsf{cert}),$$

where phase records whether each rule version η is unallocated ($\perp$), inactive, active, or closed. The bootstrap rule runs only after $\mathsf{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts, and its exact construction appears in section A-G. Each installed version stores $c = (\mathsf{state}_c, H^+_c, u, W, \widehat{B}, Z)$: the authenticated source snapshot, derived target, edit, safe language, indexed family, and checked certificate.

Definition 7 (Agent security-state invariant)**.** *For $c = (\mathsf{state}_c, H^+_c, u, W, \widehat{B}, Z)$ and resolved r, $\mathsf{AgentSec}(S; c, r)$ holds exactly when the following clauses hold.*

1) Core and schema coherence. *The state is well formed, $c = \mathsf{cert}(\eta)$, its origin derives the installed target, and every required outcome has one authenticated authority and a checked Covers_u witness.*
2) Required-outcome coherence. *$\widehat{B} = \widehat{B}^\dagger_{H_c,u} \neq \varnothing$, $W = \mathsf{Pref}(\pi_2 \widehat{B})$, and Z verifies both at state_c.*
3) Execution coherence. *$(H.G, H.T) = \mathsf{HRes}((H^+_c.G, H^+_c.T), r)$, $H.\chi = H^+_c.\chi r$, and authorization records, queue entries, return links, and checkpoints match r and advance only monotonically.*
4) Automaton coherence. *$\mathsf{prog}(\eta) \cong \mathsf{Can}(W, \pi_2 \widehat{B})$, and $\mathsf{q}(\eta)$ is its derivative after r.*
5) Rule-version coherence. *Only η is active, $\mathsf{bind}|_{X_{[\![H]\!].T}} = H.\beta$, and it owns exactly one authenticated entry and token for each current workflow call, while inactive or closed versions expose none.*

a) Atomic protected call: For submitted tool request m, set $m^\circ = \mathsf{canon}_\kappa(m)$. For x bound to rule entry j in version η, $\mathsf{VerifyToken}(h, \rho(x), j, d, m^\circ)$ verifies the call authorization service, scope, rule entry, protected action, registered source, and signed digest and requires $m^\circ = \mathsf{inv}_\rho(d)$. Set

$a = \mathsf{new}(x, d)$ if $d \notin D_\Delta$, otherwise $a = \mathsf{reuse}(x, d)$, and let $\mathsf{Use}_x(m) = \mathsf{Use}(x, j, h, m)$. Its sole state-changing transition is

$$\frac{\begin{array}{c} x \text{ enabled in } T \quad \mathsf{bind}(x) = (j, \eta) \\ \eta = H.\eta \quad \mathsf{phase}(\eta) = \mathsf{active} \\ \mathsf{VerifyToken}(h, \rho(x), j, d, m^\circ) \quad \mathsf{q}(\eta) \xrightarrow{a}_{\mathsf{prog}(\eta)} q' \end{array}}{S \xrightarrow{\mathsf{Use}_x(m)/a} S'}. \tag{12}$$

The paired update $\mathsf{HStep}((G, T), a)$ removes x, records any choice, preserves the other workflow constructors, and appends a to its branch cursor. $\mathsf{Advance}(S, x, a, q')$ changes exactly

$$\begin{aligned} (H.G, H.T) &:= \mathsf{HStep}((G, T), a), & H.\chi &:= \chi a, \\ H.\mathsf{ver} &:= \mathsf{ver} + 1, & H.\beta &:= \beta \setminus \{x\}, \\ \mathsf{bind} &:= \mathsf{bind} \setminus \{x\}, & \mathsf{q}(\eta) &:= q'. \end{aligned}$$

In addition, new appends the immutable labeled authorization record and ordered dispatch-queue item $(d, \mathsf{inv}_\rho(d))$, never an unchecked caller buffer. reuse records the protected action's stored return link without changing Δ, and all other fields remain unchanged. An authenticated Retry follows the archived link from χ to the authorization record for the same canonical request and returns any stable value without changing state.

b) Answer verification and atomic installation: At its linearization point, the trusted installation service ignores any caller-supplied replacement workflow or rule version, sets $\Theta_S(u) = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$, and re-runs $\mathsf{Derive}(\Theta_S(u))$ from the current state. For a positive derivation, $\mathsf{ReCut}(S, u, Z)$ recomputes the right-hand side of equation (11), while Verify_Z recomputes the fixed point, witnesses, canonical automaton, and both seals. Installation requires

$$\begin{aligned} \kappa = \kappa_Z, \qquad & \mathsf{ReCut}(S, u, Z) = \mathsf{cut}_Z, \\ & \mathsf{Verify}_Z(W^\dagger_{H,u}, B^\dagger_{H,u}, \widehat{B}^\dagger_{H,u}), \\ H.\eta = \eta^-, \qquad & \mathsf{phase}(\eta^+) \in \{\bot, \mathsf{inactive}\}. \end{aligned} \tag{13}$$

$\mathsf{Ready}(S, u, Y)$ holds when $Y = \mathsf{Installable}(\delta, W^\dagger, \widehat{B}^\dagger, Z)$, the current derivation is δ, and all state-dependent premises of equation (13) hold. The committing domain transaction installs $(\kappa_u, H_u, \mathcal{A}_u)$, closes η^-, activates η^+, binds every target workflow call, installs the canonical automaton at q^0, and stores the checked certificate. Only after activation does it expose $\mathcal{H}^+_u$ and emit $\mathsf{Edge}(u, [\mathcal{H}^+_u]_\cong)$; old tokens then fail, while Retry still follows its archived authorization-record link. A stale candidate emits no answer and must be recomputed from the current source snapshot.

J. Event Derivatives and Invariant Preservation

Write $A_1, \ldots, A_5$ for the five clauses of AgentSec in their stated order. The following matrix is exhaustive for state-changing transitions inside an installed registered call model. Initial registration occurs before the root safety check; later additive schema and policy growth uses the registered extension transition listed below.

Lemma 15 (Representative-independent event derivative)**.** *The guarded update induces a partial function ∂ on joint state–event orbits $\langle V, e\rangle_\cong$. If $\mathsf{AgentSec}(S; c, r)$ and $S \xrightarrow{e} S'$, the witness in table III satisfies $\mathsf{AgentSec}(S'; c', r')$ and*

$$[\mathsf{Sec}(S'; c', r')]_\cong = \partial(\langle \mathsf{Sec}(S; c, r), e\rangle_\cong).$$

Proof. All transition rules use typed equality, membership, order, canonical constructors, and authenticated payload equality. These operations are equivariant under a simultaneous private-key renaming of the state and event payload. For an event e, let $G_e(V)$ be its transition rule and $\mathsf{U}_e(V)$ its successor update when that automaton holds. For every such renaming π, $G_e(V) \iff G_{\pi e}(\pi V)$ and $\mathsf{U}_{\pi e}(\pi V) = \pi \mathsf{U}_e(V)$. Thus equal pointed orbits have the same definedness and the same successor orbit. This proves representative independence and functionality.

For new and reuse, lemmas 3, 5 and 6 establish the first two rows of the matrix. If x lies at b, HStep appends a to b's cursor, while the same atomic protected-call update appends a to the global resolved-workflow call trace χ; residual associativity gives

$$(\Gamma_b/\mathsf{raw}(\chi_b))/x = \Gamma_b/\mathsf{raw}(\chi_b a).$$

Thus the updated leaf and any later Save remain synchronized with G. For a successful edit transition, lemmas 2, 6 to 8 and 10 establish the third row for all six schema rows and registered extension. Each remaining rule changes only the fields shown in the matrix, and its row checks every invariant clause whose fields change. Thus every successor has the displayed witness and normalized update. □

Corollary 2 (Finite-prefix closure)**.** *Starting from any valid bootstrap, every finite prefix of any sequence of the event classes in table III ends in a state satisfying* AgentSec*. The sequence length is unbounded, although every individual safety-check instance and event payload is finite. For a finite word $\bar{e}$, the iterated derivative is defined on $\langle V, \bar{e}\rangle_\cong$, where one renaming acts on the initial view and the whole word.*

Proof. The base case is lemma 14, and lemma 15 supplies both the invariant-preserving inductive step and equivariance under one renaming of the remaining event suffix. □

K. Uncontrollable Events and Recovered-State Crashes

The resolved alphabet contains only commits mediated by the runtime automaton. Agent request arrival, scheduling, settlement, delivery, and crash are environmental events, but none appends a new or reuse symbol. Request arrival and scheduler choice do not change stored state until a validated transaction commits. Settlement and delivery emit modeled API observations but leave the resolved prefix unchanged. Dispatch is observable only when the oldest queued request is recorded as released, and it also leaves the resolved prefix unchanged. Therefore these uncontrollable events stutter with respect to the conditional-nonblocking plant state. When an environmental action commits a resolved workflow call, the

Event class	State update	Witness	Preservation argument
new use	Apply HStep to (G,T); append $\mathsf{new}(x,d)$ to global χ; advance ver, q; append one authorization record and canonical-request queue item; remove the current binding.	$(c, r\mathsf{new}(x,d))$	A_1: registry and schema unchanged. A_2: residual coherence. A_3: unique authorization record and ordered preparation. A_4: canonical derivative. A_5: remove exactly x's binding.
reuse use	Apply HStep to (G,T); append $\mathsf{reuse}(x,d)$ to global χ; advance ver, q; add only the return-record link; remove current binding.	$(c, r\mathsf{reuse}(x,d))$	A_1: authenticated protected action retained. A_2: residual coherence. A_3: link names an earlier authorization record and the authorization sequence is unchanged. A_4: canonical derivative. A_5: remove exactly x's binding.
Successful edit transition, six edits or registered extension	Install derived $(\kappa^+, H^+, \mathcal{A}^+)$, checked automaton and certificate; close η^-; activate η^+; publish new bindings.	(c^+_δ, ϵ)	A_1: schema fidelity and lifts. A_2: declarative–algorithmic bridge and checker. A_3: source authorization records and dispatch queue retained. A_4: install at q_ϵ. A_5: complete automaton replacement.
Verified Invalid or Reject	Emit the certificate for the current checked state; no stored-state change.	(c, r)	All clauses unchanged; γ binds the visible answer to this source snapshot.
Save	Snapshot the current residual/cursor pair in an authenticated checkpoint; advance ver.	(c, r)	A_1: immutable well-typed extension. A_2, A_4, A_5: unchanged. A_3: exactly the permitted K extension.
Preload	Allocate or replace content only in an inactive unbound rule version.	(c, r)	A_1–A_4: unchanged. A_5: inactive and unbound remain true; no authorization token is exposed.
Retry, retrieval, delivery	Emit $\mathsf{Return}(t, [v]_{\cong})$ for an authenticated stored value; no security-state change.	(c, r)	All clauses unchanged; retry additionally checks the same canonical request and authorization record link.
Denial	Return $\mathsf{deny}(x)$; no state mutation.	(c, r)	All clauses unchanged.
$\mathsf{Dispatch}(d)$	Mark exactly the oldest prepared dispatch-queue item released.	(c, r)	A_3: release order remains a prefix of preparation order. Other clauses unchanged.
Settlement	Emit $\mathsf{Settle}(d, [v]_{\cong})$, advance one authorization record phase, and fill its stable return record.	(c, r)	A_3: protected action, canonical request, creator, authorization label, and order are immutable. Other clauses unchanged.
Stale or malformed candidate	No stored-state change.	(c, r)	All clauses unchanged; the attempt is τ and emits no verdict.
Crash and recovery	Discard unfinished work and expose the same recovered stored state.	(c, r)	All clauses unchanged under the assumed crash-stable transaction boundary.

TABLE III
EXHAUSTIVE EVENT CLASSES AND PRESERVATION OF THE FIVE $\mathsf{AgentSec}$ CLAUSES.

deployment synthesizes the standard supremal controllable sublanguage.

Lemma 16 (Uncontrollable stutter)**.** *If* $\mathsf{AgentSec}(S; c, r)$*,* e *is an uncontrollable modeled event, and* $S \xrightarrow{e} S'$*, then the resolved prefix and canonical automaton state are unchanged and* $\mathsf{AgentSec}(S'; c, r)$ *holds.*

Proof. Request arrival and scheduling do not change stored state, while denial, retry, retrieval, delivery, and failure recovery preserve the recovered security state. Dispatch and settlement change only the dispatch queue, authorization-record status, or stored return record permitted by execution coherence. None changes $H.T$, $H.\chi$, the authorization sequence derived from Δ, r, or $\mathsf{q}(\eta)$, and the corresponding row of table III preserves every other invariant clause. □

The crash theorem instantiates the recovered-state interface of section III with linearizable, crash-stable domain transactions. Let D be the stored state, v the unfinished in-memory state, and $\bar{t} = t_1 \cdots t_n$ the finite set of domain transactions overlapping a failure, ordered by a linearizability witness. Linearizability and crash stability select a prefix $\bar{t}_{\le k}$ containing exactly the transactions that linearized before the crash and provide

$$\mathsf{Recover}(D, v, \bar{t}) = \mathsf{commit}_{t_k} \circ \cdots \circ \mathsf{commit}_{t_1}(D),$$

with the empty prefix interpreted as D. No recovered image contains a strict subset of one modeled atomic update. In particular, a new transaction cannot expose an authorization record without its global-trace record, branch-local cursor record, and dispatch-queue preparation. Installation cannot activate a new rule version without closing the old version and installing all target bindings.

Lemma 17 (Recovered-state crash refinement)**.** *Let* $S_0 \to S_1 \to \cdots \to S_n$ *be the modeled path whose guarded rule transitions correspond to* $t_1, \ldots, t_n$ *in the linearizability order above. If stored state* D *refines* S_0*, then recovery after the selected committed prefix* $t_1 \cdots t_k$ *refines* S_k*. At recovered-state LTS boundaries, crash is consequently a* τ *self-loop.*

Proof. Induction on k starts from D refining S_0. At each step, the complete atomic update commit_{t_i} is the guarded rule update in the corresponding row of table III, so it carries a concrete image refining S_{i-1} to one refining S_i. The recovery equation applies exactly the first k such updates and no partial update, yielding the claim. Once the LTS records the recovered image, crashing again changes only volatile state and hence becomes a self-loop. □

L. Agent-API Weak Bisimulation After Recovery

We now prove theorem 5 by exhausting the observable and silent cases. For $K = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$, $\mathsf{Current}(H) = X_{[\![H.T]\!]}$, and $I = (K, c_I, r_I)$, let $\alpha_I(I) = (K, c_I, r_I, H.\eta, H.\beta)$ and $\alpha_S(S; c, r) = (K, \mathsf{icut}(c), r, H.\eta, \mathsf{bind}|_{\mathsf{Current}(H)})$. Define $I \mathcal{B} S$ iff some c, r satisfy $\mathsf{AgentSec}(S; c, r)$ and $\alpha_I(I) = \alpha_S(S; c, r)$; write $\overset{\ell}{\Rightarrow}= \tau^* \ell \tau^*$ and $\overset{\tau}{\Rightarrow}= \tau^*$. Fix $I \mathcal{B} S$ with witness (c, r). Equality of $\alpha_I(I)$ and $\alpha_S(S; c, r)$ gives both machines the same κ, current policy, normalized execution record, authorization log, dispatch queue, stored return records, ideal projection, current partial execution, rule version, and rule-entry bindings.

a) Edit transitions: The normalization equality makes both machines form the same current $\Theta_I(u)$ and source-snapshot token $\gamma(\Theta_I(u))$. If derivation is undefined, deter-

ministic failed-check reflection makes both sides emit the same canonical Invalid orbit and take a state self-loop. If derivation exists but no declarative safe implementation exists, lemma 8 and deterministic fixed-point checking make both sides emit the same canonical Reject orbit and take a state self-loop. Otherwise the ideal machine enables its edit transition and, by lemma 8, the compiled checker accepts exactly the same edit and installs the same largest language. Compilation takes an explicit Preload transition in the compiled LTS: it records the candidate key in inactive metadata, exposes no authorization token or verdict, and preserves AgentSec. The ideal side takes its single atomic edge, and both successors normalize to the same $(\kappa_u, H_u, \mathcal{A}_u)$, ideal projection, empty prefix, active rule version, and target bindings. Deterministic allocation gives the same authorization-token bundle up to private renaming, so both expose $\mathsf{Edge}(u, [\mathcal{H}_u^+]_{\cong})$. Conversely, every successful compiled installation has passed the schema, fixed-point, certificate, and source-snapshot checks, so lemma 8 enables the matching ideal edge. This argument applies uniformly to all six execution edits and the registered extension.

b) Protected calls and submitted requests: For a submitted tool request m, both automata compute the same $m^\circ = \mathsf{canon}_\kappa(m)$. The compiled authorization-token check verifies the call authorization service, scope, workflow call, rule entry, protected action, registered source, signed digest, current rule version, and equality $m^\circ = \mathsf{inv}_\rho(d)$. The ideal automaton requires the same normalized binding and request equality. Both sides then inspect the same set D_Δ of protected actions with authorization records, so they choose the same $a \in \{\mathsf{new}(x,d), \mathsf{reuse}(x,d)\}$. By corollary 1 and lemma 6, ra is enabled by the compiled derivative exactly when $ra \in L^*$ in the ideal machine.

In the new case, both successors apply the same paired residual, append the same symbol to the global trace and branch-local cursor, advance ver, create the same protected-action-bound authorization record, and prepare the same canonical request. The compiled dispatch queue stores $(d, \mathsf{inv}_\rho(d))$, never the caller buffer, while the ideal queue stores (d, m°); the verified equality makes these records identical. In the reuse case, both successors apply the same paired update, append the same symbol that creates no authorization record to the global record and branch cursor, and bind the workflow call to the same stored return record without changing Δ or the dispatch queue. Both successors remove x's current binding; because the remaining workflow no longer contains x, their normalized current-binding components remain equal. In either case, lemma 15 re-establishes $\mathcal{B}$ with witness (c, ra).

If any workflow call, rule entry, authorization token, canonical request, rule version, branch, or automaton transition check fails, both normalized automata fail. Both machines emit $\mathsf{deny}(x)$, preserve state, and remain related.

c) Use–installation races: Use and installation serialize on the same execution and rule-version records. If a new use commits first, it changes $(G, T, \Delta, \chi, \mathsf{ver}, \mathsf{out})$, all of which are bound directly or through H by the source-snapshot seal, so the prepared installation becomes stale. If an reuse use commits first, it changes $(G, T, \chi, \mathsf{ver})$ even though Δ and the dispatch queue are unchanged, so the prepared installation again becomes stale. If installation commits first, it atomically closes η^-, activates η^+, refreshes every current rule entry and authorization token, and exposes the replacement authorization tokens only after activation. The old new or reuse use then fails the current-rule-version automaton and is denied without progress. The ideal atomic order has exactly the same two cases, so neither side admits a third race outcome.

d) Replies, releases, and silent steps: An authenticated retry, authorization-token retrieval, or result delivery leaves the security state unchanged and follows the same authenticated link on both sides to expose $\mathsf{Return}(t, [v]_{\cong})$. $\mathsf{Dispatch}(d)$ is enabled at the same dispatch-queue head and makes the same observable release update. $\mathsf{Settle}(d, [v]_{\cong})$ is enabled for the same authorization record and produces the same stable result and normalized update. Save takes matching τ-steps. Compiled preload and stale or malformed candidate attempts are implementation-only τ-steps matched by zero ideal steps because α_S omits their inactive contents and they emit no verdict. Crash is matched by a τ self-loop on each recovered-state machine by lemma 17.

Theorem 6 (Agent-API weak bisimulation after recovery, detailed)**.** *$\mathcal{B}$ is a divergence-insensitive weak bisimulation under* $\mathsf{obs}_{\mathrm{api}}$.

Proof. The least-generated rule sets in sections A-H and V and table III make the preceding cases exhaustive. For every step from either side, the corresponding case constructs a path with observation $\mathsf{obs}_{\mathrm{api}}(\ell)$ and a successor related by the witness in that matrix. The construction is symmetric because successful ideal automata and successful runtime automata were shown equivalent, while implementation-only steps preserve the common abstraction. Arbitrary repetition of silent preload, stale-candidate, Save, or crash steps is ignored, which is precisely divergence-insensitive weak matching. □

M. Executable-Artifact Evaluation

The performance script runs every sample in a fresh CPython 3.12 process and reports the median of five repetitions on a 16-vCPU AMD EPYC 7R12. The 19 paper-specific tests exercise indexed pomset resolution, the greatest fixed point, and finite fixtures for all six constructors. The executable model covers the checker core and all six constructor fixtures, and the theorem development establishes immutable schema refinement, authority-bound removal certificates, and the complete domain-wide rule-version protocol.

AI Use Acknowledgment

OpenAI Codex was used throughout the manuscript for initial prose drafting and revision, and in the executable artifact for code scaffolding and test generation. It also served as an interactive aid for literature discovery, counterexample search, and the discussion and refinement of the formal theory, including definitions, assumptions, theorem statements, and proof structure. The authors chose and directed the research, made all final theoretical and methodological decisions, independently verified the cited sources, theorem statements, proofs, code, and experimental results, and remain responsible for every claim, proof, result, and citation.

References

[1] OpenAI, "Codex App Server," Official documentation, 2026, accessed 2026-07-31. [Online]. Available: https://learn.chatgpt.com/docs/app-server.md

[2] Anthropic, "Manage sessions," Official documentation, 2026, accessed 2026-07-31. [Online]. Available: https://code.claude.com/docs/en/sessions

[3] ——, "Checkpointing," Official documentation, 2026, accessed 2026-07-31. [Online]. Available: https://code.claude.com/docs/en/checkpointing

[4] C. Wang and Y. Zheng, "Fork, explore, commit: Os primitives for agentic exploration," 2026, agentic OS Workshop 2026. [Online]. Available: https://arxiv.org/abs/2602.08199

[5] T. Wu, C. Chang, L. Cao, W. Gao, and W. Wang, "Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes," 2026. [Online]. Available: https://arxiv.org/abs/2604.28138

[6] Y. Zhang, "Agent libos: A runtime substrate for capability-controlled self-evolving llm agents," 2026. [Online]. Available: https://arxiv.org/abs/2606.03895

[7] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," 2026, coDAIM Workshop 2026. [Online]. Available: https://arxiv.org/abs/2603.20625

[8] S. Yu, D. Chong, A. Nandi, D. Soylu, J. Sun, C. D. Manning, and W. Shi, "Shepherd: Enabling programmable meta-agents via reversible agentic execution traces," 2026. [Online]. Available: https://arxiv.org/abs/2605.10913

[9] Y. Li, S. Ping, X. Chen, X. Qi, Z. Wang, Y. Luo, and X. Zhang, "AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2511.00628

[10] LangChain AI, "Use time-travel," 2026, accessed 2026-08-03. [Online]. Available: https://docs.langchain.com/oss/python/langgraph/use-time-travel

[11] S. G. Patil, T. Zhang, V. Fang, N. C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," 2024. [Online]. Available: https://arxiv.org/abs/2404.06921

[12] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4874–4886, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p4874-chang.pdf

[13] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, transactional tool use for reliable agentic workflows," 2026. [Online]. Available: https://arxiv.org/abs/2602.14849

[14] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, "Cordon: Semantic transactions for tool-using llm agents," 2026. [Online]. Available: https://arxiv.org/abs/2606.17573

[15] K. Yang, P. Li, Z. Wu, K. Xu, H. Huang, and X. Huang, "DART: Semantic recoverability for structured tool agents," 2026. [Online]. Available: https://arxiv.org/abs/2605.23311

[16] M. H. de Queiroz, J. E. R. Cury, and W. M. Wonham, "Multitasking supervisory control of discrete-event systems," *Discrete Event Dynamic Systems*, vol. 15, no. 4, pp. 375–395, 2005.

[17] R. Malik and R. Leduc, "Generalised nonblocking," in *Proceedings of the 9th International Workshop on Discrete Event Systems*, 2008, pp. 340–345.

[18] B. Finkbeiner, F. Klein, and N. Metzger, "Live synthesis," *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 443–454, 2022. [Online]. Available: https://doi.org/10.1007/s11334-022-00447-5

[19] L. Aceto, I. Cassar, A. Francalanza, and A. Ingólfsdóttir, "On runtime enforcement via suppressions," in *29th International Conference on Concurrency Theory (CONCUR 2018)*, ser. Leibniz International Proceedings in Informatics, vol. 118. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 34:1–34:17.

[20] J. A. Brzozowski, "Derivatives of regular expressions," *Journal of the ACM*, vol. 11, no. 4, pp. 481–494, 1964.

[21] R. E. Strom and S. Yemini, "Optimistic recovery in distributed systems," *ACM Transactions on Computer Systems*, vol. 3, no. 3, pp. 204–226, 1985.

[22] E. Y. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, "A survey of rollback-recovery protocols in message-passing systems," *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002.

[23] W. M. P. van der Aalst and T. Basten, "Inheritance of workflows: An approach to tackling problems related to change," *Theoretical Computer Science*, vol. 270, no. 1–2, pp. 125–203, 2002.

[24] L. Nahabedian, V. A. Braberman, N. D'Ippolito, S. Honiden, J. Kramer, K. Tei, and S. Uchitel, "Dynamic update of discrete event controllers," *IEEE Transactions on Software Engineering*, vol. 46, no. 11, pp. 1220–1240, 2020.

[25] G. Amram, S. Maoz, I. Segall, and M. Yossef, "Dynamic update for synthesized GR(1) controllers," in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 2022, pp. 786–797.

[26] Q. Burke, A. Vahldiek-Oberwagner, M. Swift, and P. McDaniel, "It's a feature, not a bug: Secure and auditable state rollback for confidential cloud applications," 2025, accepted at the 2026 IEEE Symposium on Security and Privacy. [Online]. Available: https://arxiv.org/abs/2511.13641